Department of Computer Science and Engineering,
PES University, Bangalore, India

Lecture Notes
Problem Solving With C
UE24CS151B

Lecture #1
Array in C

By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU

Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Poojari (Dean, Internal Quality Assurance Cell, PES University)

Unit #: 2

Unit Name: Counting, Searching and Sorting

Topic: Array in C

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

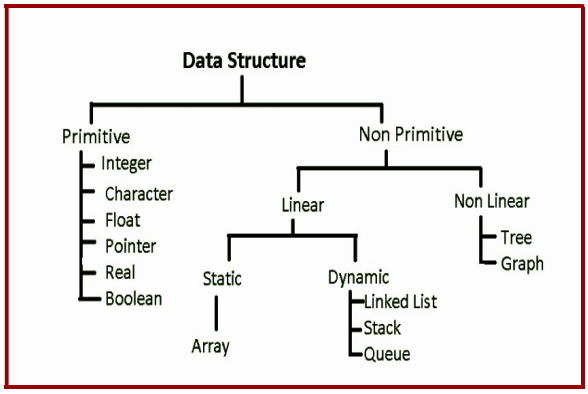
Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction to Arrays in C

Data structures are **used** to **store data** in an **organized** and **efficient** manner so that the **retrieval is efficient.** Classification of data structures in C is as below.



When solving problems, it is important to visualize the data related to the problem. Sometimes the data consist of just a single number (as we discussed in unit-1, primitive types). At other times, the data may be a coordinate in a plane that can be represented as a pair of numbers, with one number representing the x-coordinate and the other number representing the y-coordinate. There are also times when we want to work with a set of similar data values, but we do not want to give each value a separate name. For example, we want to read marks of 1000 students and perform several computations. To store marks of thousand students, we do not want to use 1000 locations with 1000 names. To store group of values using a single identifier, we use a data structure called an Array.

An array is a linear data structure, which is a finite collection of similar data items stored in successive or consecutive memory locations. Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value. It can store a fixed-size sequential collection of elements of the same type. An array is used to store a collection of data, but it is often more useful to think of an array as a collection of variables of the same type.

To create an array, define the data type and specify the name of the array followed by square brackets [] with size mentioned in it. **Example: int s[100];**

Characteristics/Properties of Arrays:

- Non-primary data or secondary data type
- Memory allocation is contiguous in nature
- Elements need not be unique.
- Demands same /homogenous types of elements
- Random access of elements in array is possible
- Elements are accessed using index/subscript
- Index or subscript starts from 0
- Memory is allocated at compile time.
- Size of the array is fixed at compile time. Returns the number of bytes occupied by the array.
- Cannot change the size at runtime.
- Arrays are assignment incompatible.
- Accessing elements of the array outside the bound will have undefined behaviour at runtime.

Types of Array

Broadly classified into two categories as

Category 1:

1. Fixed Length Array**:** Size of the array is fixed at compile time
2. Variable Length Array **-** Not supported by older few C standards. Better to use dynamic memory allocation functions than variable length array.

Category 2:

1. One Dimensional Array
2. Multi-Dimensional Array

One dimensional Array: 1D Array

- One dimensional array is also called as linear array(also called as 1-D array).
- The one dimensional array stores the data elements in a single row or column.

Array Definition:

Syntax: Data_type Array_name[Size]; // Declaration syntax

- Data_type : Specifies the type of the element that will be contained in the array
- Size: Indicates the maximum number of elements that can be stored inside the array
- Array_name: Identifier to identify a variable.
- Subscripts in array can be integer constant or integer variable or expression that yields integer
- C performs no bound checking – care should be taken to ensure that the array indices are within the declared limits

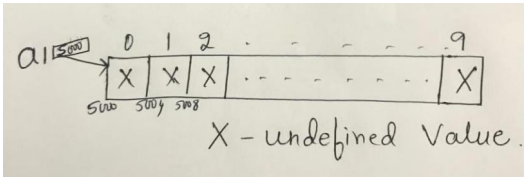
Examples:

- float average [6]: // Can contain 6 floating point elements, 0 to 5 are valid array indices
- char name[20]; // Can contain 20 char elements, 0 to 19 are valid array indices
- double x[15]; // Can contain 15 elements of type double, 0 to 14 are valid array indices.

Consider int a1[10];

Definition allocates number **of bytes in a contiguous manner** based on the size specified. All these memory locations are filled with undefined values/junk/garbages. If the starting address is 0x5000 and if the size of integer is 4 bytes, refer the diagrams below to understand this definition clearly. Number of bytes allocated = size specified*size of integer(implementation specific – in my sustem it is 4) i.e, 10*4.

printf("size of array is %d\n", sizeof(a1)); // 40 bytes



Address of the first element is called the Base address of the array. Address of i^{th} element of the array can be found using formula: **Address of i^{th} element = Base address + (size of each element * i)**

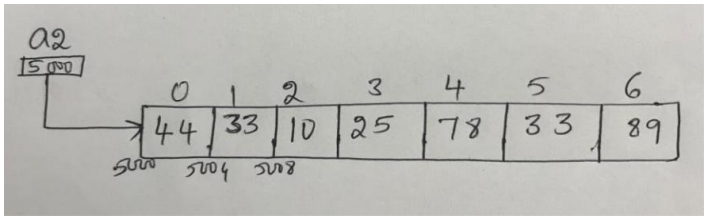
Array Initialization:

- After an array is declared it must be initialized.
- An Uninitialized array will contain undefined values/junk values.
- An array can be initialized at either compile time or at runtime

Consider int a2[ ] = {44,33,10,25,78,33,89};

Above initialization does not specify the size of the array. Size of the array is based on the number of elements stored in it and the size of type of elements stored. So, the above array occupies 7*4 = 28 bytes of memory in a contiguous manner.

printf("size of array is %d\n", sizeof(a2)); //28 bytes



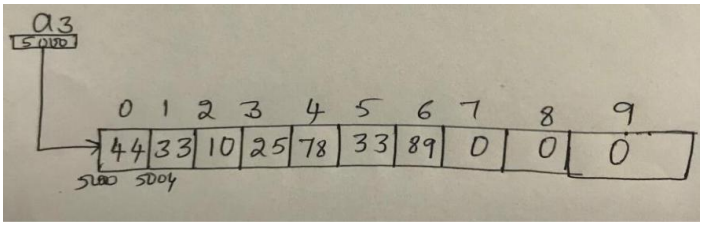
Partial Initialization:

Consider int a3[10] = {44,33,10,25,78,33,89};

Size of the array is specified. But only few elements are stored. Now, the size of the array is 10*4 = 40 bytes of memory in a contiguous manner. **All uninitialized memory locations in the array are filled with default value 0.**

printf("size of array is %d\n", sizeof(a3)); //40 bytes

printf("%p %p\n",a3, &a3); // Must be different. But shows same address



Compile time Array initialization:

Compile time initialization of array elements is same as ordinary variable initialization. The general form of initialization of array is,

Syntax: data-type array-name[size] = { list of values };

```
int marks[4]={ 67, 87, 56, 77 }; // integer array initialization
float area[5]={ 23.4, 6.8, 5.5 }; // float array initialization
intarr[] = {2, 3, 4};
int marks[4]={67,87,56,77,5} //undefined behavior
```

Variable length array:

Variable length arrays are also known as runtime sized or variable sized arrays. The size of such arrays is defined at run-time.

Example:

```
void fun(int n)
{
 int arr[n];
 // ......
}
int main()
{
  fun(6);
}
```

The variable length arrays are always unsafe. This dynamically creates stack memory based on data size, which can cause overflow of stack frame.

Designated initializers (C99):

- It is often the case that relatively few elements of an array need to be initialized explicitly; the other elements can be given default values:

  int a[15] = {0,0,29,0,0,0,0,0,0,7,0,0,0,0,48};

- So, we want element 2 to be 29, 9 to be 7 and 14 to be 48 and the other values to be zeros. For large arrays, writing an initializer in this fashion is tedious.
- C99's designated initializers

```
int a[15] = {[2] = 29, [9] = 7, [14] = 48};
```

- Each number in the brackets is said to be a designator.
- Besides being shorter and easier to read, designated initializers have another advantage: the order in which the elements are listed no longer matters. The previous expression can be written as:

```
int a[15] = {[14] = 48, [9] = 7, [2] = 29};
```

Designators must be constant expressions:

- If the array being initialized has length n, each designator must be between 0 and n-1.
- if the length of the array is omitted, a designator can be any non-negative integer.
- In that case, the compiler will deduce the length of the array by looking at the largest designator.

```
int b[] = {[2] = 6, [23]=87};
```

- Because 23 has appeared as a designator, the compiler will decide the length of this array to be 24.
- An initializer can use both the older technique and the later technique.

```
int c[10] = {5, 1, 9, [4] = 3, 7,2,[8]=6};
```

Runtime initialization: Using a loop and input function in c

Example:

```
inta[5];
for(int i=0;i<5;i++)
{
        scanf("%d",&a[i]);
 }
```

Traversal of the Array:

Accessing each element of the array is known as traversing the array.

Consider int a1[10];

- How do you access the 5th element of the array? **a1[4]**
- How do you display each element of the array?

```
int i;
for(i = 0; i<10;i++)
  printf("%d\t",a1[i]);//Using the index operator [ ].
//Above code prints undefined values as a1 is just declared.
```

Consider int a2[ ] = {12,22,44,14,77,911};

```
int i;
for(i = 0; i<10;i++)
  printf("%d\t",a2[i]);//Using the index operator [ ].
```

Above code prints initialized values when i is between 0 to 5. When i becomes 6, a2[6] is outside bound as the size of a2 is fixed at compile time and it is 6*4(size of int is implementation specific) = 24 bytes

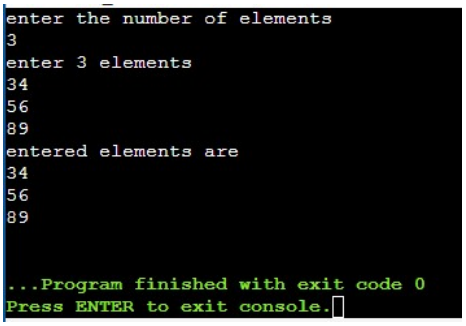
Anytime accessing elements outside the array bound is an undefined behavior.

Coding Example_1: Let us consider the program to read 10 elements from the user and display those 10 elements.

```
#include<stdio.h>
int main()
{       int arr[100];  int n;
        printf("enter the number of elements\n");
        scanf("%d",&n);
        printf("enter %d elements\n",n);
        int i = 0;
```

```
while(i<n)
{       scanf("%d",&arr[i]);            i++;
}
printf("entered elements are\n");
i = 0;
while(i<n)
{       printf("%d\n",arr[i]);          i++;
}
return 0;
}
```

Output:



Coding Example_2: Let us consider the program to find the sum of the elements of an array.

```
int arr[] = {-1,4,3,1,7};    int i;  int sum = 0;
int n = sizeof(arr)/sizeof(arr[0]);
for(i = 0; i<n; i++)
{       sum += arr[i];  }
printf("sum of elements of the array is %d\n", sum);
```

Happy Coding using Arrays!!

Department of Computer Science and Engineering,
PES University, Bangalore, India

Lecture Notes
Problem Solving With C
UE24CS151B

Lecture #2
Two Dimensional Array in C

By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU

Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Poojari (Dean, Internal Quality Assurance Cell, PES University)

Unit #: 2

Unit Name: Counting, Searching and Sorting

Topic: Two Dimensional Array in C

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

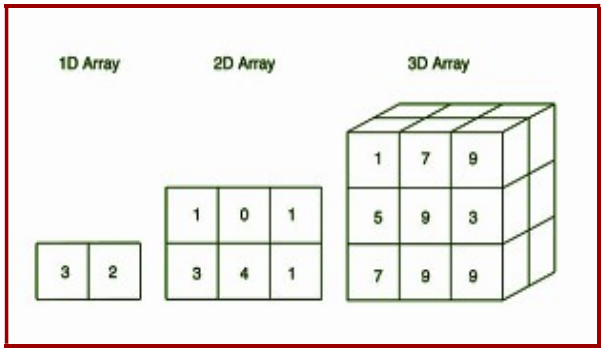
Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction to 2D Arrays in C

A multi-dimensional array is an array with more than one level or dimension. It might be 2-Dimensional and 3-Dimensional and so on.



1D array may be used to store single dimension data or information. When we want to manipulate the arrays by rearranging the elements by using functions like reshape, squeeze and so on, which may be very challenging to visualize using a 1D array. Then, 2D array can come into play we may have to store complex data which has rows and columns. Likewise, we may want to store three dimensions of data as well. Then we go for 3D array and so on.

Consider the example of storing marks of 5 students in some particular subject A. We can define, int A_marks[ ] = {90,95,99,100,89};

Consider the example of storing marks of 5 students in 3 different subjects A, B and C.

Version-1:

```
int A_marks[ ] = {90,95,99,100,89};
int B_marks[ ] = {95,90, 91,99, 94};
int C_marks[ ] = {91,92,93,95,100};
```

Version-2:

```
int student_1_marks[ ] = {90, 95, 91};
int student_2_marks[ ] = {95, 90, 92};
int student_3_marks[ ] = {99, 91, 93};
int student_4_marks[ ] = {100, 99, 95};
int student_5_marks[ ] = {89, 94, 100};
```

Now, rather than storing marks this way in One - Dimensional Array, we can use Two - Dimensional array as we have **two dimensions involved** in this particular example: student and subject marks

Two – Dimensional Array

It is treated as an array of arrays. Every element of the array must be of same type as arrays are homogeneous. Hence, every element of the array must be an array itself in 2D array.

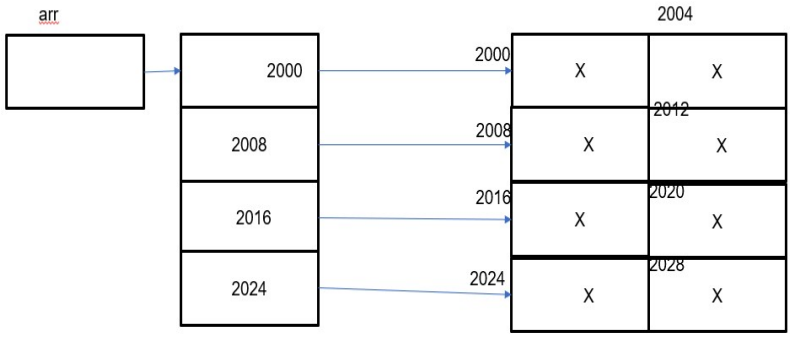
Let us consider the above example of storing 5 students marks in three different subjects.

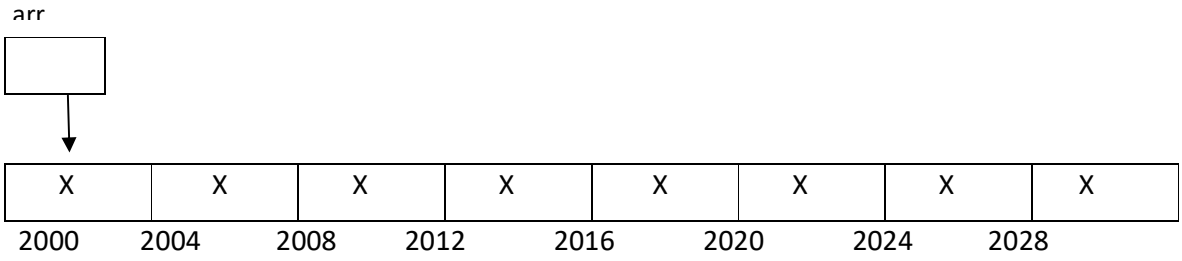
int marks[5][3] = {{90, 95, 91}, {95, 90, 92}, {99, 91, 93}, {100, 99, 95}, {89, 94, 100} };

Definition of a 2D Array:

data_type array_name[size_1][size_2];

int arr[4][2]; // Allocate 8 contiguous memory locations





If the size of the integer is 4 bytes, 8 contiguous locations allocated

Initialization of a 2D Array:

Compiler knows the array size based the array elements and allocates memory

data_type array_name[size_1][size_2] = {elements separated by comma};

int arr[][] = {11,22,33,44,55,66}; // Error. Column size is compulsory

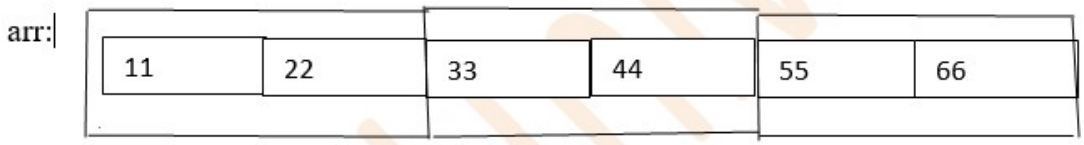
Without knowing the number of columns in each row – it is impossible to locate an element in 2D array.

int arr[3][2] = {{11,22},{33,44},{55,66}};

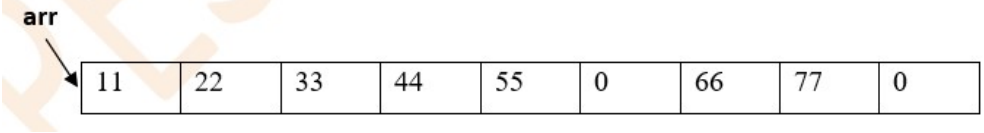
int arr[3][2] = {11,22,33,44,55,66};

// Allocate 6 contiguous memory locations and assign the values

int arr[][2] = {11,22,33,44,55,66};



int arr[][3] = {{11,22,33},{44,55},{66,77}}; **//Partial initialization**



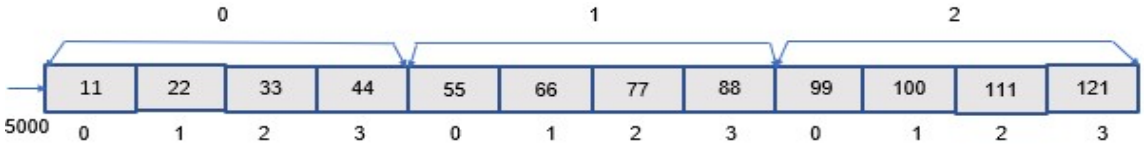
Internal Representation of a 2D Array

As two - dimensional array itself is an array, elements are stored in contiguous memory locations. And since elements are stored in contiguous memory locations, there are two possibilities such as **Row Major Ordering & Column Major Ordering**

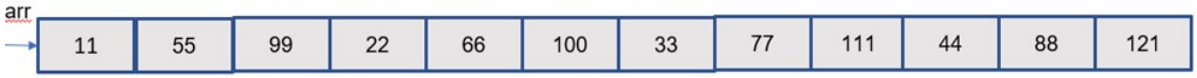
Consider,

int arr[3][4] = {{11, 22, 33, 44}, {55, 66, 77, 88}, {99, 100, 111, 121}};

Row major Ordering: All elements of one row are stored followed by all elements of the next row and so on.



Column Major Ordering: All elements of one column are stored followed by all elements of the next column and so on.



Generally, systems support Row Major Ordering.

Address of an element in a 2D Array

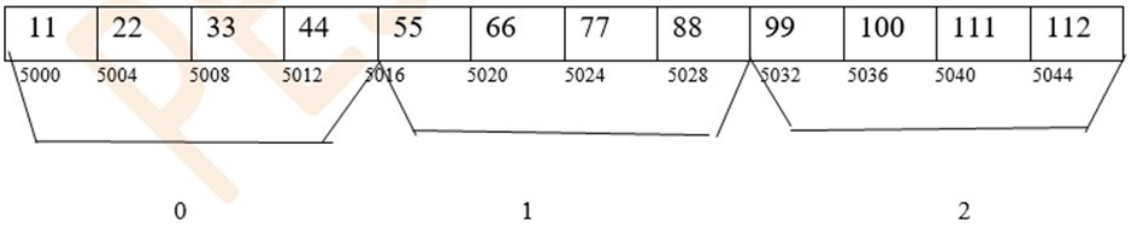
Address of A[i][j] = Base_Address +((i * No. of columns in every row) + j)*size of every element;

Consider, int arr[3][4] = {{11, 22, 33, 44}, {55, 66, 77, 88}, {99, 100, 111, 121}};

If the size of integer is 4 bytes in the system and the base address is 5000, then address of arr[1][2] can be found by 5000+((1*4)+2)*4 = 5000+6*4 = 5000+24 = 5024.

Consider, int arr[3][4] = {11, 22, 33, 44, 55, 66, 77, 88, 99, 100, 111, 121};

If the size of integer is 4 bytes in the system and the base address is 5000, then address of arr[1][5] can be found by 5000+((1*4)+5)*4 = 5000+9*4 = 5000+36 = 5036.



Think about finding the address of a[0][5] and a[1][3]. Is this same as a[5][0] and a[3][1] respectively?

Coding Example: Reading and displaying a 2D Array

```
// code to read n*m elements
int a[100][100];
int n; int m;
printf("enter row num and column numbers\n");
 scanf("%d %d",&n,&m);
printf("Enter %d elements",n*m);
for(int i = 0;i<n;i++)     // n – row index          // m – column index
{
        for(int j= 0;j<m;j++)
        {
                scanf("%d",&a[i][j]);
        }
}
// code to display n*m elements
printf("entered elements are\n");
for(int i = 0;i<n;i++)
{
        for(int j= 0;j<m;j++)
        {
                printf("%d\t",a[i][j]);
        }
        printf("\n");
}
```

Happy Coding using 2D Arrays!!

Department of Computer Science and Engineering,

PES University, Bangalore, India

Lecture Notes
Problem Solving With C
UE24CS151B

Lecture #3
Problem Solving using Arrays

By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU

Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)

Unit #: 2

Unit Name: Counting, Sorting and Searching

Topic: Problem Solving using Arrays

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Solve the below programs using C Arrays.

Link for Solution:
https://drive.google.com/file/d/1ki5RyURIrY8OxB04OEb9bUPR2606Rv3D/view?usp=drive_link

Level-1: Banana

1. Given the array, build an **application to find the sum of all elements of the array.**
   int arr1[ ] = {5, 10, 15, 20, 25};

2. Seven friends decided to have a race in the park. After the race, they recorded their running times in seconds: **[12, 15, 10, 18, 9, 14, 11]**
   Implement the logic in C to **find out the running time for a fastest runner**?

3. Develop a C Application to **Count the number of elements in the given array** and print the count.
   int arr1[ ] = {5, 10, 15, 20, 25, 12, 99, 56, 33, 29, 10};

4. Build an application in C to **find the count of even and odd integers** in a given array.
   int arr1[ ] = {5, 10, 15, 20, 25, 12, 99, 56, 33, 29, 10};

5. Given an array of integers, count how many numbers are prime.
   int arr[] = {2, 3, 4, 5, 10, 17, 19, 22, 25};
   Expected output: Count of prime numbers: 5

6. Given an array, count how many numbers are positive and how many are negative.
   int arr[] = {-5, 10, -15, 20, -25, 12, -99, 56, 33, -29, 10};
   Expected output:
   Count of positive numbers: 6
   Count of negative numbers: 5

7. Given an array and a number X, count how many numbers are greater than X and print them.
   int arr[] = {5, 10, 15, 20, 25, 12, 99, 56, 33, 29, 10};
   int x = 20;
   Expected output:
   Count of numbers greater than 20: 5
   Numbers greater then 20 are - 25, 99, 56, 33, 29

8. Emma wanted to find out which stickers were repeated in her collection. She carefully arranged all the stickers and noted down their numbers in sequence:
   [12, 25, 30, 12, 45, 60, 25, 30, 75, 90, 45]. Now, she faced a challenge: **Can you figure out how many stickers appeared more than once?**

9. Given an array, count how many numbers end with a specific digit X(e.g., 5).
   int arr[] = {5, 15, 25, 30, 45, 50, 60, 75};
   X = 5
   Expected output: Count of numbers ending with 5: 5

10. A young programmer, Alex, found a magical book that contained an ancient spell to reverse time. The spell was hidden in a sequence of numbers, and to unlock its power, Alex needed to reverse the sequence. Help Alex to decode the spell by implementing an application using C.

Level-2: Orange

11. Write a program that checks and prints **if the given array of integers is a Palindrome or not**. Receive all inputs from the user.
    [1,3,2,2,3,1] -> Palindrome
    [1,3,2,3,1] -> Palindrome
    [10,32,25,32,10] -> Palindrome
    [1,4,6,3,1] ->Not A Palindrome

12. Write a Program to create an Array of n integers and check if the array is "Really Sorted" , "Sorted","Not Sorted"
    Example #1 : [1,2,5,7,10] -> Really Sorted
    Example #2 : [1,2,5,5,10] -> Sorted
    Example #3 : [1,12,5,45,10] -> Not Sorted

13. Given the array of n integers, count the **number of Unique Elements in the Array(not repeated elements)** and **store these elements in a new array.**
    int arr[] = {5, 7, 3, 4, 5, 6, 8, 9, 10, 3};
    Expected Input and Output:
    Given Array -> [5,7,3,4,5,6,8,9,10,3]
    New array -> [7,4,6,8,9,10]
    Number of Unique Elements -> 6

14. Write a C Program that should rotate the given array by N positions at the left.
    Expected Input and Output:
    Input: [5,4,7,3]
    If N = 2
    After left Rotation -> [7,3,5,4]

15. Given a list of *N* integers, representing height of mountains. Find the height of the tallest mountain and the second tallest mountain. Handle edge cases by displaying appropriate messages.
    Example #1 : [4,7,6,3,1]
    7 -> height of the tallest mountain
    6 -> height of the second tallest mountain

Level-3: Jackfruit

16. Write a C program to input N integers to the array and find the median of sorted array of integers. Median is calculated using the below formulae:

    - If N is odd:

    $$\text{Median} = \text{Element at } \left(\frac{N}{2}\right)^{\text{th}} \text{ position}$$

    - If N is even:

    $$\text{Median} = \frac{\text{Element at } \left(\frac{N}{2} - 1\right) \text{ position} + \text{Element at } \left(\frac{N}{2}\right) \text{ position}}{2}$$

17. Abhi is a salesman. He was given two types of candies, which he is selling in N different cities. In his excitement, Abhi wrote down the prices of both the candies on the same page and in random order instead of writing them on different pages. Now he is asking for your help to find out if the prices he wrote are valid or not. You are given an array A of size 2N. Find out whether it is possible to split A into two arrays, each of length N, such that both arrays consist of distinct elements.

18. Given an array A of size N, your task is to **find and print all the peak elements in the array.** A peak element is one that is strictly greater than its neighboring elements. For the first and last elements, only consider their single adjacent element. If no peak element exists in the array, print -1.

19. Develop an application that **calculates and prints the largest sum of two Adjacent Elements in the array. Display appropriate message for handling the edge cases.**
    Example #1 : [1,4,3,7,1] ->10
    Example #2 : [5,7,1,5,2] ->12

20. Our James has some students in his coding class who are practicing problems. Given the difficulty of the problems that the students have solved in order, help the James identify if they are solving them in non-decreasing order of difficulty. **Non-decreasing means that the values in an array is either increasing or remaining the same, but not decreasing**. That is, the students should not solve a problem with difficulty d1, and then later a problem with difficulty d2, where d1>d2. Output “Yes” if the problems are attempted in non-decreasing order of difficulty rating and “No” if not.

Happy Coding using Arrays!!!

Department of Computer Science and Engineering,
PES University, Bangalore, India

Lecture Notes
Problem Solving With C
UE24CS151B

Lecture #4
Pointers in C

By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU

Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)

Unit #: 2

Unit Name: Counting, Searching and Sorting

Topic: Pointers in C

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

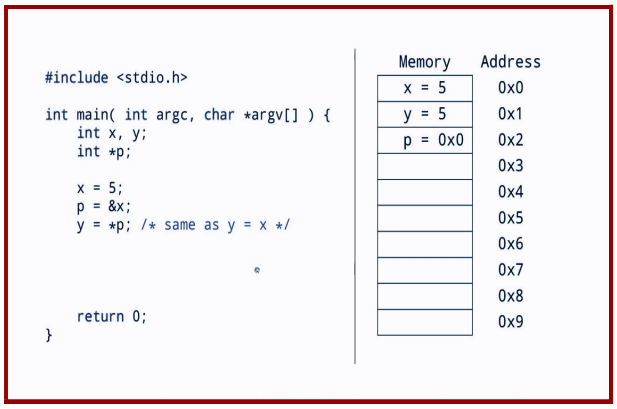
- Pointer is a variable which contains the address. This address is the location of another object in the memory.
- Pointers can be used to access and manipulate data stored in memory.
- Pointer of particular type can point to address any value of that particular type.
- Size of pointer of any type is same /constant in that system.
- Not all pointers actually contain an address

  Example: NULL pointer // Value of NULL pointer is 0.

Pointer can have three kinds of contents in it.

1. The address of an object, which can be de referenced.
2. A NULL pointer
3. Undefined value // If p is a pointer to integer, then – int *p;

Note: A **pointer** is a **variable** that **stores** the **memory address** of **another variable** as **its value. The address of the variable we are working with is assigned to the pointer.**



We can get the value of the variable the pointer currently points to, by dereferencing pointer by using the * operator. When used in declaration like int* ptr, it creates a pointer variable. When not used in declaration, it act as a dereference operator w.r.t pointers

Pointer Definition:

Syntax: Data-type *name;

Example: int *p; // Compiler assumes that any address that it holds points to an integer type.

p= ∑ // Memory address of sum variable is stored into p.

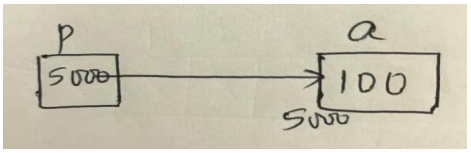
Coding Example_1:

int *p; // p can point to anything where integer is stored. int* is the type. Not just int.

int a = 100;

p=&a;

printf("a is %d and *p is %d", a,*p);



Coding Example_2: Pointer pointing to an array

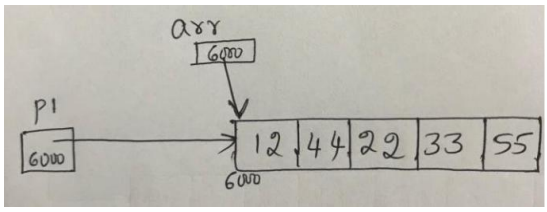
Now, int arr[] ={12,44,22,33,55};

int *p1 = arr; // same as int *p1;

p1 = arr; // same as int *p1; p1 = &arr[0];

int arr2[10];

arra2 = arr; // **Arrays are assignment incompatible. Compile time Error**



Pointer Arithmetic:

Below arithmetic operations are allowed on pointers

- Add an int to a pointer
- Subtract an int from a pointer
- Difference of two pointers when they point to the same array.

Note:

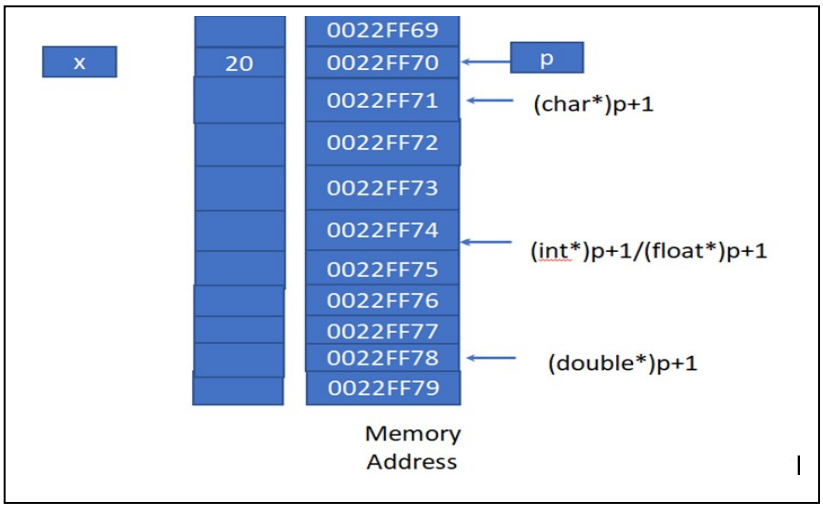
- **Integer is not same as pointer.**
- We get warning when we try to compile the code where integer is stored in variable of int* type.

Coding Example_3:

```
int arr[ ] = {12,33,44};
int *p2 = arr;
printf("before increment %p %d\n",p2, *p2);          //  Some address     12
p2++;   //same as p2 = p2+1
        // This means 5000+sizeof(every element)*1 if 5000 is the base address
         //increment the pointer by 1. p2 is now pointing to next location.
printf("after increment %p %d\n",p2, *p2);  // 33
```

Coding Example_4: Example on Pointer Arithmetic

```
int *p, x = 20;
p = &x;
printf("p     = %p\n", p);
printf("p+1 = %p\n", (int*)p+1);
printf("p+1 = %p\n", (char*)p+1);
printf("p+1 = %p\n", (float*)p+1);
printf("p+1 = %p\n", (double*)p+1);
```



Sample output:

p = 0022FF70

p+1 = 0022FF74

p+1 = 0022FF71

p+1 = 0022FF74

p+1 = 0022FF78

Coding Example_5:

```
int main()
{
        int *p;
        int a = 10;
        p = &a;
```

```
        printf("%d\n",(*p)+1);        // 11 ,p is not changed
        printf("before *p++ %p\n",p);          //address of p
        printf("%d\n",*p++);          // same as *p   and then p++          i.e 10
        printf("after *p++ %p\n",p);
        //address incremented by the size of type of value stored in it
        return(0);
}
```

Coding Example_6:

```
int main()
{
        int *p;
        int a = 10;
        p = &a;
        printf("%d\n",*p);//10
        printf("%d\n",(*p)++);// 10 value of p is used and then value of  p is incremented
        printf("%d\n",*p); // 11
        return 0;
}
```

Array Traversal using pointers

Coding Example_7: Try all these versions after writing the diagrams properly.

```
    int arr[] ={12,44,22,33,55};
    int *p3 = arr;
    int i;
```

Version 1: Index operator can be applied on pointer. Array notation

```
    for(i = 0;i<5;i++)
        printf("%d \t",p3[i]); // 12 44 22 33 55
    // every iteration added i to p3 .p3 not modified
```

Version 2: **Using pointer notation**

```
for(i = 0;i<5;i++)
        printf("%d\t",*(p3+i));          //  12   44      22      33      55
// every iteration i value is added to p3 and content at that address is printed.
// p3 not modified
```

Version 3:

```
for(i = 0;i<5;i++)
        printf("%d \t",*p3++);             // 12     44     22     33     55
// Use p3, then increment, every iteration p3 is incremented.
```

Version 4: **Undefined behavior if you try to access outside bound**

```
for(i = 0;i<5;i++)
        printf("%d      \t",*++p3);     //  44   22      33      55      undefined value
// every iteration p3 is incremented.
```

Version 5:

```
for(i = 0;i<5;i++)
                printf("%d \t",(*p3)++); // 12 13 14 15 16
// every iteration value at p3 is used and then incremented.
```

Version 6:

```
for(i = 0;i<5;i++,p3++)
        printf("%d \t",*p3); // 12  44  22  33 55
// every iteration value at p3 is used and then p3 is incremented.
```

Version 7: **p3 and arr has same base address of the array stored in it. But array is a constant pointer. It cannot point to anything in the world.**

```
for(i = 0;i<5;i++)
        printf("%d       \t", *arr++);     // Compile Time Error
```

One-Dimensional Arrays and Pointers

An array during compile time is an actual array but degenerates to a constant pointer during run time. Size of the array returns the number of bytes occupied by the array. But the size of pointer is always constant in that particular system.

Coding Example_8:

```
int *p1;
float *f1 ;
char *c1;
printf("%d%d%d ",sizeof(p1),sizeof(f1),sizeof(c1)); // Same value for all
int a[] = {22,11,44,5};
int *p = a;
a++;     // Error constant pointer
p++;     // Fine
p[1] = 222; //  allowed
a[1] = 222 ; // Fine
```

Note: If variable i is used in loop for the traversal, a[i], *(a+i), p[i], *(p+i), i[a], i[p] are all same.

Differences between an Array and a Pointer

1. The sizeof operator:
   - sizeof(array) returns the amount of memory used by all elements in array
   - sizeof(pointer) only returns the amount of memory used by the pointer variable itself

2. The & operator:
    - &array is an alias for &array[0] and returns the address of the first element in array
    - &pointer returns the address of pointer

3. String literal initialization of a character array – Will be discussed in detail in next lecture
    - char array[] = “abc” sets the first four elements in array to ‘a’, ‘b’, ‘c’, and ‘\0’
    - char *pointer = “abc” sets pointer to the address of the “abc” string (which may be stored in read-only memory and thus unchangeable)
    - Pointer variable can be assigned a value whereas array variable cannot be.

4. Pointer variable can be assigned a value whereas array variable cannot be.

```
int a[10];
int *p;
p=a; //allowed
a=p; //not allowed
```

5. An arithmetic operation on pointer variable is allowed.

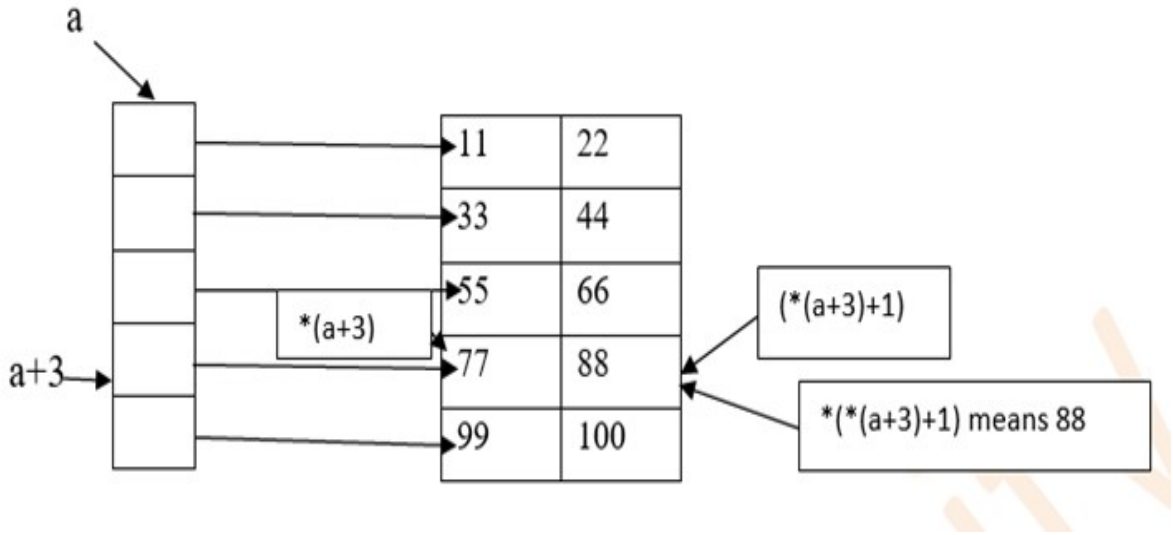
```
int a[10];
int *p;
p++; /*allowed*/
a++; /*not allowed*/
```

Two-Dimensional Arrays and Pointers

Consider int a[5][2] = {{11,22}, {33,44}, {55,66}, {77,88}, {99,100}};

```
printf("Using array, accessing the elements of the array");
printf("%d\n",a[3][2]);          // 99   array notation
printf("%d\n",a[3][1]);          // 88    array notation
printf("%d\n",*(*(a+3)+1)); // 88       // pointer notation
```

Array name is a pointer to a row. The expression a + 3 is a pointer to the third row. *(a + 3) becomes a pointer to the zeroth element of that row. Then +1 becomes a pointer to the next element in that row. To get the element in that location, dereference the pointer.



In General, (a + i) points to ith 1-D array

//int *p1 = a;// incompatible type. Warning during compilation

// If above statement is fine, what happens when p1[7] is accessed and p1[2][1] is accessed

int (*p)[2] = a; // **p is a pointer to an array of 2 integers**

Since subscript([ ]) have higher precedence than indirection(*), it is necessary to have parentheses for indirection operator and pointer name. Here the type of p is 'pointer to an array of 2 integers'.

```
printf("using pointer accessing the elements of the array")
printf("%d\n",p[3][1]);          // 88   array notation
printf("%d\n",*(*(p+3)+1)); // 88      pointer notation
```

Note : The pointer that points to the 0th element of array and the pointer that points to the whole array are totally different.

Coding Example_9: Illustration of the size of pointer to an array of items.

```
int arr[4][3]={33,44,11,55,88,22,33,66,99,11,80,9};
//assigning array to a pointer
int *p=arr;                    //gives warning
//printf("%d",p[7]);  //looks fine, not a good idea.
//printf("\n%d\n",p[2][1]);          //error..Column size not available to p.
//create a pointer to an array of integers
int (*p)[3]=arr;
printf("%d\n",p[2][1]);          // Array notation
printf("%d\n",*(*(p+2)+1));    //Pointer notation
printf("%d\n",sizeof(p));      //size of pointer
printf("%d %d\n",sizeof(*p),sizeof(arr));       //size of the array pointing to 0th row and size of the entire 2D array
```

Happy Coding using Pointers!!

Department of Computer Science and Engineering,
PES University, Bangalore, India

Lecture Notes
Problem Solving With C
UE24CS151B

Lecture #5
Array of Pointers in C

By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU

Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)

Unit #: 2

Unit Name: Counting, Searching and Sorting

Topic: Array of Pointers in C

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

An array of pointers is a collection where each element is a pointer that stores the address of another variable (typically of the same type). It is commonly used in scenarios where multiple variables or memory locations need to be dynamically referenced. Array and pointers are closely related to each other. The name of an array is considered as a pointer, i.e., the name of an array contains the address of an element.

Array of Pointer Definition:

int *arr[5]; // An array of 5 pointers to integers

Applications:

Array of pointers to integers offers several benefits in terms of **memory efficiency, flexibility, and computational performance.**

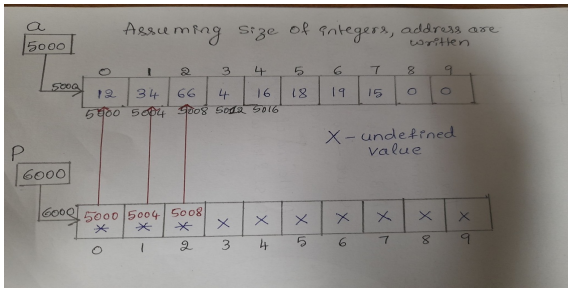
- Saves memory, especially when dealing with sparse data structures.
- Useful when data sizes vary, such as in jagged arrays.
- Supports implementation of complex data structures like trees and graphs.
- Makes handling **pointers to pointers** more structured.
- Used in **hash tables** where each index points to a linked list.

Coding Example_1:

```
#include<stdio.h>
int main()
{
        int i;
        int a[10] = {12,34,66,4,16,18,19,15};
        int *p[10]; // p is created with the intention of storing an array of addresses
        // Think about the size of p???
        p[0] = &a[0];
        p[1] = &a[1];
        p[2] = &a[2];
        for(i = 0 ; i<4;i++){
```

```
        printf("%p %p\n",p[i],&a[i]); // same address for all 3 pairs except the last iteration as we have not  assigned.
    }
    // printing array elements using p
    for(i = 0 ; i<4;i++) {
            printf("%d\t",*p[i]); // dereference every element of the array of pointer
    }
    return 0;
}
```

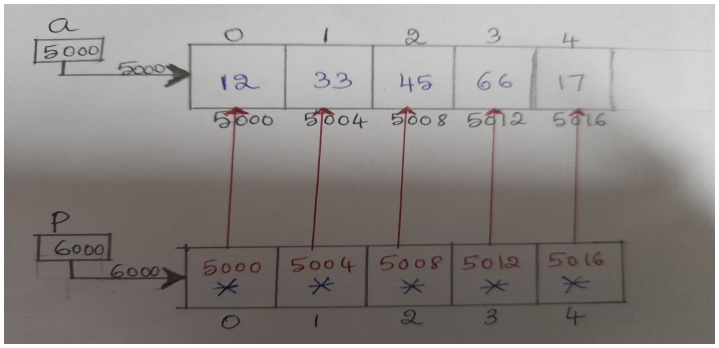
Note: *arr[i] dereferences the pointer to access the actual value.



Coding Example_2: Program to print the elements of the array using array of pointers

```
#include<stdio.h>
int main()
{           int a[5] = {12,33,45,66,17}; // a is an array
            printf("Using original array\n");
            for(int i = 0;i<5;i++)
            {     printf("%d ",a[i]);     }
            printf("\n");
            int *p[5]; // p is an array of pointers.
            for(int i = 0;i<5;i++)
            {    p[i] = &a[i]; // address of a[i] stored in p[i]    }
            printf("Using array of pointers\n");
            for(int i = 0;i<5;i++)
            {     printf("%d ",*p[i]);
                  // content at p[i] is displayed. All elements are displayed
            }
```

```
        printf("\n");
        return 0;
}
```



Happy Coding using Array of pointers !!

Department of Computer Science and Engineering,
PES University, Bangalore, India

Lecture Notes
Problem Solving With C
UE24CS151B

Lecture #6
Functions in C

By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU

Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)

Unit #: 2

Unit Name: Counting, Searching and Sorting

Topic: Functions in C

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction to Functions

In case we want to repeat the set of tasks or instructions, we may have two options:

- Use the same set of statements every time we want to perform the task
- Create a function to perform that task, and just call it every time when we need to perform that task. This is usually a good practice and a good programmer always uses functions while writing code in C.

Functions **break large computing tasks into smaller ones** and enable people to build on what others have done instead of starting from scratch. In programming, **Function is a subprogram to carry out a specific task**. A function is a self-contained block of code that performs a particular task. Once the function is designed, it can be **treated as a black box**. The inner details of operation are invisible to rest of the program. Functions are useful because of following reasons:

- To **improve the readability** of code.
- **Improves the reusability** of the code, same function can be used in any program rather than writing the same code from scratch.
- **Debugging of the code would be easier** if we use functions, as errors are easily traced.
- **Reduces the size of the code**, redundant set of statements are replaced by function calls.

Types of functions

C functions can be broadly classified into two categories:

Library Functions

Functions which are defined by C library. Examples include printf(), scanf(), strcat() etc. We just need to include appropriate header files to use these functions. These are already declared and defined in C libraries.

User-defined Functions

Functions which are defined by the developer at the time of writing program. Developer can make changes in the implementation as and when he/she wants. It reduces the complexity of a big program and optimizes the code, supporting Modular Programming Paradigm. In order to make use of user defined functions, we need to understand three key terms associated with functions: **Function Definition, Function call and Function Declaration.**

Function Definition

Each function definition has the form

```
return_type function_name(parameters)   // parameters optional
{
        // declarations and statements
}
```

A function definition should have a return type. The return type must match with what that function returns at the end of the function execution. If the function is not designed to return anything, then the type must be mentioned as void. Like variables, function name is also an identifier and hence must follow the rules of an identifier. Parameters list is optional. If more than one parameter, it must be comma separated. Each parameter is declared with its type and parameters receive the data sent during the function call. If function has parameters or not, but the parentheses is must. Block of statements inside the function or body of the function must be inside { and }. There is no indentation requirement as far as the syntax of C is considered. For readability purpose, it is a good practice to indent the body of the function.

Function definitions can **occur in any order in the source file** and the source program can be split into multiple files. One must also notice that **only one value can be returned from the function, when called by name.** There can also be side effects while using functions in c.

```
int sum(int a,  int b)
{               return a+b;             }
int decrement(int y)
{               return y-1;             }
```

```
void disp_hello()       // This function doesn't return anything
{                   printf("Hello Friends\n");      }
double use_pow(int x)
{                   return pow(x,3);        // using pow() function is not good practice.
}
int fun1(int a, int)                // Error in function definition.
{ }
int fun2(int a, b)                  // Invalid Again.
{  }
```

Function Call

Function-name(list of arguments); // semicolon compulsory and Arguments depends on the number of parameters in the function definition

A function can be called by using **function name followed by list of arguments** (if any) enclosed in parentheses. The function which calls other function is known to be Caller and the function which is getting called is known to be the Callee. **The arguments must match the parameters in the function definition in it's type, order and number**. **Multiple arguments must be separated by comma. Arguments can be any expression in C. Need not be always just variables. A function call is an expression. When a function is called, Activation record is created**. **Activation record is another name for Stack Frame**. It is composed of:

- Local variables of the callee
- Return address to the caller
- Location to store return value
- Parameters of the callee
- Temporary Variables

The order in which the arguments are evaluated in a function call is not defined and is determined by the calling convention(out of the scope of this notes) used by the compiler. It is left to the compiler Writer. **Always arguments are copied to the corresponding parameters**. Then the control is transferred to the called function. Body of the function gets executed. When all the statements are executed, callee returns to the caller. OR when there is

return statement, the expression of return is evaluated and then callee returns to the caller. If the return type is void, function must not have return statement inside the function.

Coding Example_1:

```
#include<stdio.h>
int main()
{
        int x = 100;
        int y = 10;
        int answer = sum(x,y);
        printf("sum is %d\n",answer);
        answer = decrement(x);
        printf("decremented value is %d\n",answer);
        disp_hello();
        double ans = use_pow(x);
        printf("ans is %lf\n",ans);
        answer = sum(x+6,y);
        printf("answer is %d\n", answer);
        printf("power : %lf\n", use_power(5));
        return 0;
}
```

Function Declaration/ Prototype

All functions must be declared before they are invoked. The function declaration is as follows.

return_type Function_name (parameters list); // semicolon compulsory

A function may or may not accept any argument. A function may or may not return any value directly when called by name. There are different type of functions based on the arguments and return type.

- Function without arguments and without return value
- Function without arguments and with return value
- Function with arguments and without return value
- Function with arguments and with return value

The parameters list must be separated by commas. The parameter names do not need to be the same in declaration and the function definition. The types must match the type of parameters in the function definition in number and order. Use of identifiers in the declaration is optional. When the declared types do not match with the types in the function definition, compiler will produce an error.

Parameter Passing in C

Parameter passing is always by Value in C. Argument is copied to the corresponding parameter. The parameter is not copied back to the argument. It is possible to copy back the parameter to argument only if the argument is l- value. Argument is not affected if we change the parameter inside a function.

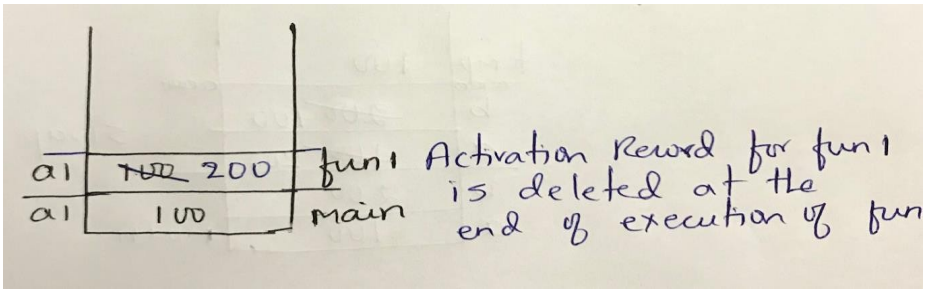
Types of parameters: Actual Parameter/Arguments and Formal Parameters

Actual parameters are values or variables containing valid values that are passed to a function when it is invoked or called. **Formal parameters** are the variables defined by the function that receives values when the function is called.

Coding Example_2:

```
void fun1(int a1); // declaration
void main()
{
        int a1 = 100;
        printf("before function call a1 is %d\n", a1); // a1 is 100
        fun1(a1);        // call
        printf("after function call a1 is %d\n", a1);    // a1 is 100
        return 0;
}
void fun1(int a1)
{       printf("a1 in fun1 before changing %d\n", a1);        //100 a1 = 200;
        printf("a1 in fun1 after changing %d\n", a1);//200
}
```

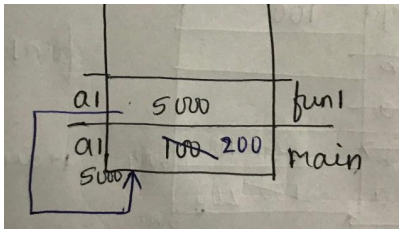
a1 has not changed in main function. The parameter a1 in fun1 is a copy of a1 from main function. Refer to the below diagram to understand activation record creation and deletion to know this program output in detail.



Think about this Question: Is it impossible to change the argument by calling a function? Yes – by passing the l-value

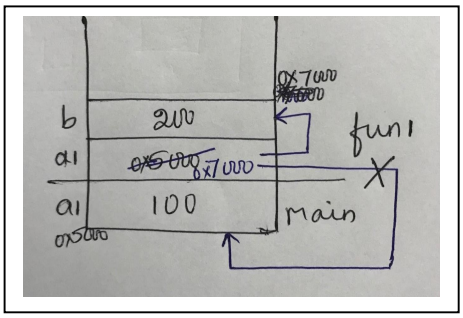
Coding Example_3:

```
void fun1(int *a1);
int main()
{       int a1 = 100;
        printf("before function call a1 is %d\n", a1); // 100
        fun1(&a1);      // call
        printf("after function call a1 is %d\n", a1);    // 200
        return 0;
}
void fun1(int *a1)
{       printf("*a1 in fun1 before changing %d\n", *a1);      //100
        *a1 = 200;
        printf("*a1 in fun1 after changing %d\n", *a1);       //200
}
```



Coding Example_3:

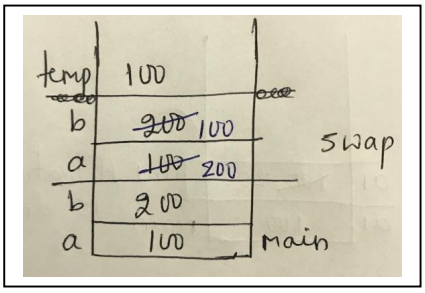
```
void fun1(int *a1);
int main()
{
        int a1 = 100;
        printf("before function call a1 is %d\n", a1); // 100
        fun1(&a1);      // call
        printf("after function call a1 is %d\n", a1);    // 100
        return 0;
}
void fun1(int *a1)
{
        int b = 200;
        printf("*a1 in fun1 before changing %d\n", *a1);      //100
        a1 = &b;
        printf("*a1 in fun1 after changing %d\n", *a1);       //200
}
```



Coding Example_4: To swap two numbers and test the function

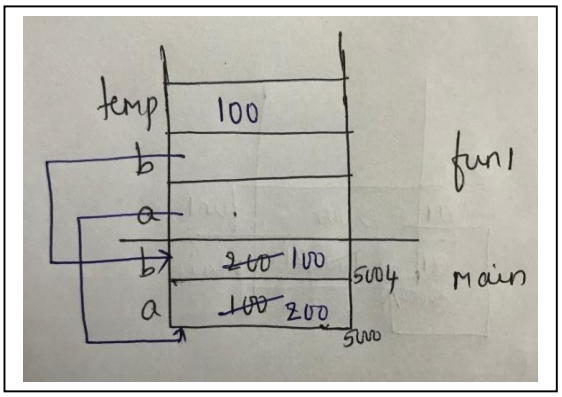
Version – 1: Is this right version?

```
void swap(int a, int b)
{
        int temp = a; a = b; b = temp;
}
int main()
{       int a = 100; int b = 200;
        printf("before call a is %d and b is %d\n", a, b);       // a is 100 and b is 200
        swap(a, b);
        printf("after call a is %d and b is %d\n", a, b);        // a is 100 and b is 200
        return 0;
}
```



Version 2:

```
void swap(int *a, int *b)
{
        int temp = *a;
        *a = *b;
        *b = temp;
}
int main()
{       int a = 100;
        int b = 200;
        printf("before call a is %d and b is %d\n", a, b);      // a is 100 and b is 200
        swap(&a, &b);
        printf("after call a is %d and b is %d\n", a, b);       // a is 100 and b is 200
        return 0;
}
```



The keyword - return

Function must do what it is supposed to do. It should not do something extra. For Example, refer to the below code.

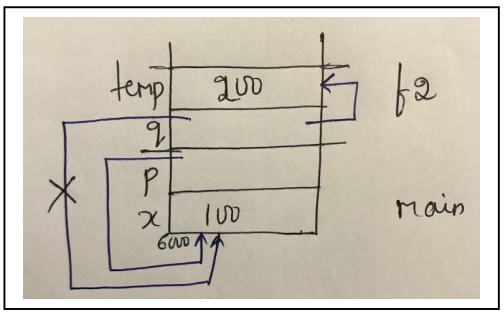
```
int add(int a, int b)
{       return a+b;    }
```

The function add() is used to add two numbers. It should not print the sum inside the function. We cannot use this kind of functions repeatedly if it does something extra than what is required. Here comes the usage of return keyword inside a function. **Syntax: return expression;**

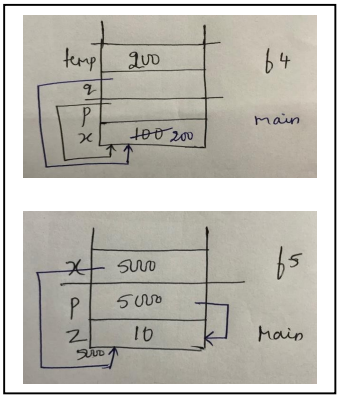
In 'C', function returns a single value. The **expression of return is evaluated and copied to the temporary location by the called function.** This **temporary location does not have any name. If the return type of the function is not same as the expression of return in the function definition, the expression is cast to the return type before copying to the temporary location**. The calling function will pick up the value from this temporary location and use it.

Coding Example_5:

```
void f1(int); void f2(int*); void f3(int); void f4(int*);int* f5(int* ); int* f6();
int main()
{       int x = 100;
        f1(x);
        printf("x is %d\n", x); // 100
        double y = 6.5;
        f1(y);  // observe what happens when double value is passed as argument to integer parameter?
        printf("y is %lf\n", y); //        6.500000
        int *p = &x; // pointer variable
        f2(p);
        printf("x is %d and *p is %d\n", x, *p);        // 100 100
        f3(*p);
        printf("x is %d and *p is %d\n", x, *p);        // 100 100
        f4(p);
        printf("x is %d and *p is %d\n", x, *p, p);
        int z= 10;
        p =f 5(&z);     printf("z is %d and %d\n", *p, z);      // 10     10
        p = f6();       printf("*p is %d \n", *p);
        return 0;
}
void f1(int x)
{       x = 20;     }
void f2(int* q)
{
        int temp = 200;
        q = &temp;
}
void f3(int t)
{       t = 200;    }
```

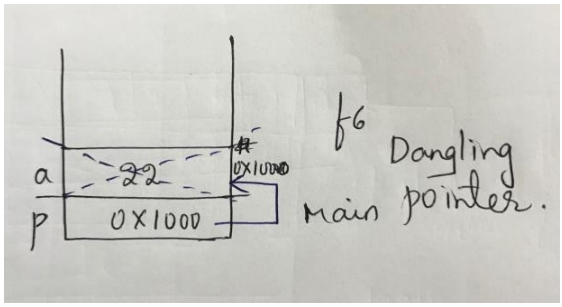


```
void f4(int* q)
{
        int temp = 200;
        *q = temp;
}
int* f5(int* x)
{       return x;    }
```



When there is function call to f6, Activation Record gets created for that and deleted when the function returns. p points to a variable which is not available after the function execution. This problem is known as Dangling Pointer. The pointer is available. But the location it is not available which is pointed by pointer. Applying dereferencing operator(*) on a dangling pointer is always undefined behaviour.

```
int* f6()
{
        int a = 22;       // We should never return a pointer to a local variable
        return &a;        //        Compile time Error
}
```



When there is function call to f6, Activation Record gets created for that and deleted when the function returns. p points to a variable which is not available after the function execution. This problem is known as Dangling Pointer. The pointer is available. But the location it is not available which is pointed by pointer. Applying dereferencing operator(*) on a dangling pointer is always undefined behaviour.

The keyword - void

void is an **empty data type that has no value**. We mostly use void data type in functions when we don't want to return any value to the calling function. We also use void data type in pointer like "void *p", which means, pointer "p" is not pointing to any non void data types . **void * acts as generic pointer.** We will be using void *, when we are not sure on the data type that this pointer will point to. We can use void * to refer either integer data or char data. **void * should not be dereferenced without explicit type casting.**

Note: We use void as an argument to function, when the function does not accept any argument.

Happy Coding using Functions!!!

Department of Computer Science and Engineering,
PES University, Bangalore, India

Lecture Notes
Problem Solving With C
UE24CS151B

Lecture #7
Arrays and Functions

By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU

Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)

Unit #: 2

Unit Name: Counting, Searching and Sorting

Topic: Arrays and Functions

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

An array is a a data structure that stores multiple values of the same type in contiguous memory locations, enabling efficient indexing and manipulation. **Functions** in C allow modular programming by encapsulating logic into reusable blocks, improving code organization and reusability. When combined, arrays and functions enable efficient data processing **without unnecessary duplications**. In C, care must be taken to avoid exceeding array bounds, as C does not perform automatic boundary checks. Overall, combining arrays and functions in C enhances **modularity, efficiency, and structured learning**.

Passing 1D array to a function

When you pass an array to a function, actually passes a **pointer to its first element** rather than the entire array. This means that **any modifications made to the array inside the function will affect the original array.** Different ways to Pass a 1D Array to a Function are as follows:

1. **Passing an Array with its Size**
2. **Using const to Prevent Modification**
3. **Using pointer notation**

Let us discuss these using some of the coding examples.

Coding Example_1: Since the name of an array acts as a pointer to its first element, you can pass the array directly to a function.

```
#include <stdio.h>
void printArray(int arr[], int size)  // Passing the array with its size
{
   for (int i = 0; i < size; i++) {
      printf("%d ", arr[i]);
   }
}
```

```
int main() {
   int numbers[] = {1, 2, 3, 4, 5};
   int size = sizeof(numbers) / sizeof(numbers[0]);
   printArray(numbers, size); // Passing array to function
   return 0;
}
```

Note:

- arr[] in the function is equivalent to int *arr, meaning it holds a reference to the array.
- The actual values in numbers are not copied; only the address of the first element is passed.
- Arrays in C do not carry size information, so we must explicitly pass it to prevent **out-of-bounds** errors.
- printArray function can modify the elements of the array. If yiyou want to ensure that the function does **not modify** the array, use the `const` keyword.

Coding Example_2:

```
void printArray(const int arr[], int size) {
   for (int i = 0; i < size; i++) {
      printf("%d\t ", arr[i]);
   }
}
```

Note: Use const, when the function is only **reading** values from the array and **not modify** it.

Coding Example_3:

```
void printArray(int *arr, int size) { ... }
```

Coding Example_4: Create functions to read n elements into an array and display n elements of the array. Also include a function to find the sum of all the elements of the array.

Version 1: n is a global variable

```
#include<stdio.h>
void read_array(int[]);
int n;  // global variable
void display_array(int[]);
int find_sum(int[]);
int main()
{
        int a[100];
        printf("how many elements u want to add\n");
        scanf("%d",&n);
        printf("enter %d elements\n",n);
        read_array(a);
        printf("entered elements are\n");
        display_array(a);
        printf("\nsum is %d\n",find_sum(a));
        return 0;
}
void read_array(int a[])
{
        for(int i= 0; i<n;i++)
        {               scanf("%d",&a[i]);      }
}
void display_array(int a[])
{
        for(int i= 0; i<n;i++)
        {               printf("%d\t",a[i]);      }
}
int find_sum(int a[])
{
        int sum = 0;
        for(int i= 0; i<n;i++)
```

```
        {               sum = sum+a[i];         }
        return sum;
}
```

Version 2: As the array becomes pointer at runtime, finding the size of the passed argument to any function is same as finding the size of the pointer.

```
void read_array(int a[])
{
        printf("inside read_array sizeof a is %d\n",sizeof(a));
                        //4 bytes or constant value for any pointer
        for(int i= 0; i<n;i++)
        {               scanf("%d",&a[i]);      }
}
```

Think!!! -> If n is local to main function, can other functions access n? It throws Compile time Error. So use version3 code if n is local to main().

Version 3: As n is local to main function, send this to functions as an argument

```
#include<stdio.h>
void read_array(int[],int);
void display_array(int[],int);
int find_sum(int[],int);
void increment(int a[],int n);
int main()
{
        int a[100];
        int n;  // Local variable n
        printf("how many elements u want to add\n");
        scanf("%d",&n);
        printf("enter %d elements\n",n);
        printf("sizeof a is %d\n",sizeof(a));
```

```
        read_array(a,n);
        printf("entered elements are\n");
        display_array(a,n);
        printf("\nsum is %d\n",find_sum(a,n));
        increment(a,n);
        printf("array with updated elements are\n");
        display_array(a,n);
        return 0;
}
void read_array(int a[],int n)
{
        for(int i= 0; i<n;i++)
        {               scanf("%d",&a[i]);      }
}
void display_array(int a[],int n)
{
        for(int i= 0; i<n;i++)
        {       printf("%d\t",a[i]);      }
}
int find_sum(int a[],int n)
{
        int sum = 0;
        for(int i= 0; i<n;i++)
        {        sum = sum+a[i];        }
        return sum;
}
```

Version 4: Using the pointer in the parameter makes more sense as array become pointer at runtime during function call.

```
#include<stdio.h>
void read_array(int*,int);
```

```
void display_array(int*,int);
int find_sum(int*,int);  // observe this
int main()
{       …    }
void read_array(int *a,int n) // this too
{       …    }
void display_array(int *a,int n)
{       …    }
int find_sum(int *a,int n)
{       …     }
```

Note: Functions like display_array() and find_sum() should not be allowed to make any changes to the array. But allowed in the above code.

```
void display_array(int *a,int n)
{       a[4] = 8989;      // allowed    }
```

Version 5: Create a pointer to constant integer and pass this to a function.

```
#include<stdio.h>
void read_array(int*,int);
void display_array(const int*,int);
int find_sum(const int*,int);
int main()
{       …    }
void read_array(int *a,int n)
{
void display_array(const int *a,int n)
{       a[4] = 8989;    }   //compiletime Error
int find_sum(const int *a,int n)
{       …     }
```

Passing a 2D Array to a function

When passing a **2D array to a function**, a pointer to the first element is passed meaning changes inside the function affect the original array. The number of **columns must be specified** in the function parameter and this **ensures that the compiler correctly interprets the memory layout of the array.**

Coding Example_5: Read and display 2D array using functions.

Consider the below client code.

```
void read(int (*a1)[],int,int);
void display( int (*a1)[],int,int);
int main()
{
        int a[100][100];        int m1,n1;
        printf("enter the order of a\n");
        scanf("%d%d",&m1,&n1); // user entered 3 5
        printf("enter the elements of a\n");
        read(a,m1,n1);
        printf("elements of A are\n");
        display(a,m1,n1);
        return 0;
}
```

Following cases to be considered while writing the definitions for functions.

Case1: Declaration and definition of the function with no column size

```
void read(int[][],int m,int n);      //Compiletime error
void display(int[][],int m,int n);
```

Case 2: Declaration and definition of the function with column size not same as what is given in the declaration of the array in the client code

```
void read(int[][3],int m,int n);      // warning but code works
void display(int[][3],int m,int n);
```

Case 3: Declaration and definition of the function with column size same as what is given in the declaration of the array in the client code

```
void read(int[][100],int m,int n);      // fine.
void display(int[][100],int m,int n);
// But can we say void read(int[][n],int m,int n ) ? Think !!!!
```

Case 4: If the order of parameters is changed as below in declaration and definition of the function, we need to change the client code. Changing the client code is not a good programmer's habit. Because interface cannot be changed. Implementation can change. So refer to case 5.

```
void read(int m,int n,int a[][n]);
void display(int m,int n,int a[][n]);
```

Case 5: Forward declaration

```
void read(int n;int[][n],int m,int n);
void display(int n; int[][n],int m,int n);
```

Coding Example_6: Function using the forward declaration.

```
void read(int n;int a[][n],int m,int n) {
        int i,j;
        for(i = 0;i<m;i++)
                for(j = 0;j < n;j++)
                {       printf("enter the element");
                        scanf("%d",&a[i][j]);
                }
}
void display(int n; int a[][n],int m,int n){
        int i,j;
        for(i = 0;i<m;i++)
        {
```

```
        for(j = 0;j < n;j++)
        {                       printf("%d\t",a[i][j]);         }
        printf("\n");
    }
}
```

Case 6: Array degenerates to a pointer at run time. So think about using pointer to an array as a parameter.

```
        void read(int n; int(*)[n],int m,int n);
        void display(int n; int(*)[n],int m,int n);

void read(int n; int (*a)[n],int m,int n)
{
    int i,j;
    for(i = 0;i<m;i++)
        for(j = 0;j < n;j++)
        {
            printf("enter the element");
            scanf("%d",&a[i][j]);
        }
}
void display(int n; int (*a)[n],int m,int n){
    int i,j;
    for(i = 0;i<m;i++)
    {   for(j = 0;j < n;j++)
        {                       printf("%d\t",a[i][j]);         }
        printf("\n");
    }
}
```

Happy Coding using Array with Functions!!!

Department of Computer Science and Engineering,
PES University, Bangalore, India

Lecture Notes
Problem Solving With C
UE24CS151B

Lecture #8
Problem Solving using Functions, Arrays and Pointers

By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU

Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)

Unit #: 2

Unit Name: Counting, Searching and Sorting

Topic: Problem Solving using Functions, Arrays – 1D, 2D and Pointers

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Solve the below programs using Functions, Arrays and Pointers.

Link for Solution:
https://drive.google.com/file/d/1HeQTdNZhWm35FpOKopamVbkfyhMvVp6A/view?usp=drive_link

Level-1: Banana

1. John wants to calculate and display the area of a circular garden with radius 5.5 meters. Use a function to find the area.

2. Emma is learning about factorials in her math class. She wants to find the factorial of a whole number n. Create a function to calculate it. Display appropriate message if the input is not valid.

3. Mark wants to check if the entered integer is a prime number for his math homework. Write a function to determine if it's prime. Display appropriate message for valid and invalid inputs.

4. Lisa is travelling to the US and needs to convert temperatures from Celsius to Fahrenheit. Create a function to convert 30°C to Fahrenheit. Display the converted value in the client.

5. Seven friends decided to have a race in the park. After recording their running times in seconds: [12, 15, 10, 18, 9, 14, 11], find the fastest runner's time.

6. A grocery store recorded the prices of 8 different fruits: [45, 60, 35, 25, 55, 70, 85, 30]. Write a function to find the most expensive fruit's price.

7. Sarah bought 6 items from a store with prices: [120, 85, 200, 35, 40, 75]. Create a function to calculate the total amount she spent. Show the total amount spent by Sarah on the terminal.

8. A teacher recorded the scores of 5 students in a test: [85, 92, 78, 90, 88]. Implement a function to calculate the average score. Display the average.

9. A coach recorded the heights (in cm) of 8 basketball players: [185, 192, 188, 195, 190, 187, 182, 193]. Implement a function to find the second tallest player's height.

10. Emily is a weather analyst who records the daily temperatures of a city for a week at different times of the day (morning, afternoon, evening, and night). She stores the temperatures in a 2D array and wants to find the highest temperature recorded during the week. Write a function that takes this 2D array as input and returns the maximum temperature.

Level-2: Orange

11. A shopkeeper, Mr. Smith, keeps track of his daily sales for five products over seven days in a 2Darray. He wants to **find out the total sales for each product** over the week. Write a **function that takes the 2D array as input and modifies a 1D array** containing the total sales of each product.

12. Write a function in **C to convert a decimal number to a binary number**. Expected output when the test code is run is as below:
    Input any decimal number: 8
    The Binary value is : 1000
    Input any decimal number: -8
    The Binary value is : -1000

13. Write a program to **remove the duplicate elements from a sorted array**. Fill the implementation of remove_duplicates function.
    Expected output: [1, 2, 3, 4, 5, 6]
    Client code is as below:
    ```
    int arr[] = {1, 1, 2, 2, 3, 4, 4, 5, 6, 6};
    int n = sizeof(arr)/sizeof(arr[0]);
    remove_duplicates(arr, n);
    ```

14. Implement a function that takes the array and the array size. This function must **calculate and display the largest sum of two Adjacent Elements (Neighbours) in the array. If the array size is less than 2, display appropriate message.**
    Example #1 : [1,4,3,7,1] ->10
    Example #2 : [5,7,1,5,2] ->12

15. Given an array A of length N, implement a function **to find the element which repeats maximum number of times and update the corresponding frequency_count**. In case of a tie, choose the smaller element.
    Expected output:
    Given array => {2,2,2,1,1,1,1,2,2,31,32,32,32};
    The element 2 repeats maximum number of times
    Frequency of this element is 5

16. Emma visited a grocery store for fresh supplies. There are N items in the store where the ith item has a freshness value A_i and cost Bi_i. Emma has decided **to purchase all the items having a freshness value greater than equal to X**. Find the **total cost of the groceries Emma buys**.

Client code is as below.

```
int a[] = {15,67, 45, 60};
int b[] = {10,90, 100, 150};
int n = sizeof(a) / sizeof(a[0]);
int x = 20; // freshness level expected by Emma
int t_cost = find_total_cost_freshness(a, b, n, x);
printf("Total cost of items above freshness level %d is %d\n", x, t_cost);
```

Fill the implementation of find_total_cost_freshness to get the expected output.
Expected output => Total cost of items above freshness level 20 is 340

17. A teacher, Mrs. Patel, maintains a 2D array containing the scores of five students in four subjects. She wants to **compute the average score of each student** to determine their performance. Write a **function that takes this 2D array as input and updates the 1D array** with the average score of each student.
    Given, int marks[5][4] = { {80, 85, 78, 90}, {70, 75, 88, 85},
    {90, 92, 89, 94}, {60, 65, 70, 68}, {85, 88, 82, 86} };
    Expected output:

    ```
    Average marks of each student:
    Student 1: 83.25
    Student 2: 79.50
    Student 3: 91.25
    Student 4: 65.75
    Student 5: 85.25
    ```

18. Alice, a data scientist, is analyzing a **4×4** dataset and wants to find the **sum of all diagonal elements** in a square matrix. Implement a function that takes an **n×n** matrix and returns the **sum of its main diagonal elements. Display the sum of diagonal elements in the client code.**

Level-3: Jackfruit

19. You are given two integer arrays A and B of size N and M respectively. You need to merge these two arrays using a function. Print the merged array using another function.

20. David is working on a game where he represents a chessboard as an 8×8 2D array, with 0 representing white squares and 1 representing black squares. He wants to rotate the board 90 degrees clockwise to generate a new pattern. Write separate functions to rotate the 2D array and print the new board configuration respectively.

```
Original Chessboard:
0 1 0 1 0 1 0 1
1 0 1 0 1 0 1 0
0 1 0 1 0 1 0 1
1 0 1 0 1 0 1 0
0 1 0 1 0 1 0 1
1 0 1 0 1 0 1 0
0 1 0 1 0 1 0 1
1 0 1 0 1 0 1 0
```

```
Rotated Chessboard (90 degrees clockwise):
1 0 1 0 1 0 1 0
0 1 0 1 0 1 0 1
1 0 1 0 1 0 1 0
0 1 0 1 0 1 0 1
1 0 1 0 1 0 1 0
0 1 0 1 0 1 0 1
1 0 1 0 1 0 1 0
0 1 0 1 0 1 0 1
```

21. Emma, a restaurant manager, records daily sales for different sections of her restaurant in a 5×4 sales matrix (5 sections, 4 days). She wants to identify which section had the highest total sales. Write separate functions to do the following.

    A. Function that takes a sales data from the user for each day for each section as a 2D array (sales matrix) of size M×N.

    B. Function that returns the index of the row (section) with the highest total sum.

22. A group of explorers discovers a sunken ship filled with treasure. The ocean floor is represented as a **5x5 grid**, where each cell contains a number representing the amount of gold in that location. Some locations have traps (-1 value).
    Consider,

```
int ocean[5][5] = {{5, 3, -1, 4, 10},
        {2, 6, 9, -1, 8},
        {-1, 7, 12, 15, 1},
        {3, -1, 4, 5, 2},
        {11, 8, 6, -1, 9}
    };
```

    Write functions to do the following.

- Find the **total treasure** collected if the explorers move from the top-left corner (0,0) to the bottom-right corner (4,4) while avoiding traps.

  Expected output:

  Total Treasure Collected: 130

- Identify the cell with the highest gold. Specify the row and column number.

  Expected output:

  Highest gold is 15 at cell (3, 4)

Explore, Code, and Innovate with Arrays and Functions – Happy Coding!!

Department of Computer Science and Engineering,
PES University, Bangalore, India

Lecture Notes
Problem Solving With C
UE24CS151B

Lecture #9
Problem Solving using Array of Pointers

By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU

Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)

Unit #: 2

Unit Name: Counting, Searching and Sorting

Topic: Problem Solving using Array of Pointers

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Solve the below programs using Array of Pointers and Functions.

Link for Solution:
https://drive.google.com/file/d/1GQPsSrv-0IPI9bpV-7D1Inb1QueKGX6G/view?usp=drive_link

Level-1: Banana

1. John is a mathematician who loves to work with collection of integers. He has an integer array of size N, and wants to calculate the **sum of all the numbers in that array.** However, he decides to use an **array of pointers to access the elements of the array** as he believes that, it will help him **understand memory management better**. Develop a C program which makes John happy by taking N integers from the user and find the sum of elements using the array of pointers.

2. Mrs. Smith is a teacher who wants to find the average height of her students. She measures the heights of N students and stores them in an array. To make her program more efficient, she decides to use an array of pointers to access the heights. **Implement a C function that receives the array of pointers and help Smith to calculate the average height of her students. Test this function the client code.**

3. David has two unique integer values, and he wants to store these in an array of pointers whose size is 2 blocks. **Swap them using an array of pointers using a function**. Print the results before and after calling the function.

4. Ryan is a programmer who has an array of pointers to array of integers. He wants to count how many even numbers are stored in this array. Write a C program to help Ryan count and display the number of even numbers in the array. He uses function **int countEvenNumbers(int *arr[], int size)** to count the even numbers. Implement the given function to make sure the client is calling the function with the same interface.

5. Alex is a mathematician who wants to **calculate the product of all the numbers stored in an array of pointers to very long integers.** He believes using an array of pointers will make the program more efficient. Write a C program to help Alex calculate the product of all the numbers in the array. She uses **long long as return type for helper function to find the product.**

Level-2: Orange

6. Sarah is a data analyst who has an integer array of size N. She wants to find the maximum and minimum values in the array. To make her program more efficient, she decides to use an array of pointers to access the integers. Help Sarah **find the maximum and minimum integer in the array using an array of pointers to integers by sending this array to findMinMax function.**
   Prototype of the function is as below with min and max as two variables in the infterface code.
   void findMinMax(int *arr[], int size, int *min, int *max);
   Function call: findMinMax(ptrArr, n, &min, &max);

7. Mike is a mathematician who wants to calculate the sum of the squares of N numbers. He decides to use an array of pointers to access the numbers and perform the calculation. Implement the C code to help Mike **calculate the sum of the squares of N numbers** using an array of pointers. Prototype: **int calculateSumOfSquares(int *arr[], int size);**

8. Bob is a programmer who has an array of pointers that point to N numbers stored in the array. He wants to **reverse the order of these numbers and then print the reversed list** of integers using an array of pointers to to make the program more efficient. Write a C function implementation to do this and test this function in the client code.
Prototype of function => void reverseArray(int *arr[], int size);

9. Emily is a mathematician who wants to **calculate the product of all prime numbers stored in an array of pointers to integers**. Implement the C function to help Emily calculate the product of all prime numbers in the array using array of pointers.
Prototype of function => int calculateProductOfPrimes(int *arr[], int size)

10. Alice is a programmer who has two fixed integer values, A and B. She wants to calculate their GCD (Greatest Common Divisor) and LCM (Least Common Multiple).
Client code is as below.

```
int a, b;
printf("Enter two integers (A B): ");
scanf("%d %d", &a, &b);
int *ptrArr[2] = {&a, &b};
int gcdResult, lcmResult;
calculateGCDAndLCM(ptrArr, &gcdResult, &lcmResult);
printf("GCD: %d\nLCM: %d\n", gcdResult, lcmResult);
```

Fill the implementation of calculateGCDAndLCM so that we get the expected output.
Expected output:

```
C:\Users\Dell>a
Enter two integers (A B): 23
4
GCD: 1
LCM: 92

C:\Users\Dell>a
Enter two integers (A B): 5 15
GCD: 5
LCM: 15
```

Level-3: Jackfruit

11. During a severe storm that knocked out power to Seoul University, Professor Ji-hoon's electronic grade book became corrupted. The only working computer had limited RAM and could only process data through pointer arrays. A group of students trapped in the dark library had to:

- Calculate the total marks using candlelight.
- Find the highest score to determine the scholarship recipient.
- Identify the count of failing students (scores <50) needing extra classes

Input :

int storm_grades[] = {68, 42, 95, 33, 57}

Output:

Emergency Results:
Total Marks: 295
Top Student Score: 95
Count of Students Needing Help: 2

12. When the Han River overflowed during monsoon season, librarian Mrs. Park had 30 minutes to save rare books from the rising waters. The library's flood control system showed:
    - 3 floors with 5 sections each
    - Sensors (1=dry, 0=flooded)

    Using an ancient DOS-based system requiring pointer arrays, she needed to:
    a) Find safe sections to move books to
    b) Identify worst-hit floor

    Input:

```
sensors[3][5] = {
        {1,0,1,1,0},
        {0,0,0,1,1},
        {1,1,1,0,0}
    };
```

    Output :

    Crisis Report:
    Safe Sections: 8
    Most Flooded Floor: 2

Master Array of Pointers and Functions!

Department of Computer Science and Engineering,
PES University, Bangalore, India

Lecture Notes
Problem Solving With C
UE24CS151B

Lecture #10
Storage Classes

By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU

Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)

Unit #: 2

Unit Name: Counting, Searching and Sorting

Topic: Storage Classes

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

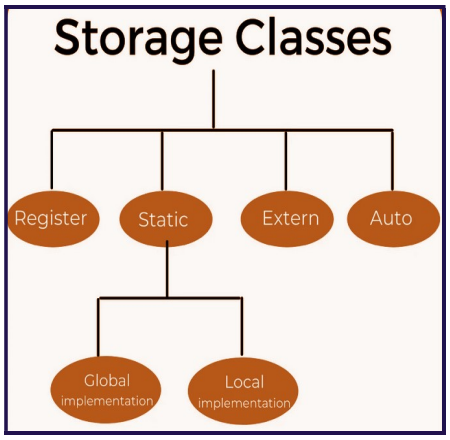
Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

Storage Classes are used to describe the features of a variable/function. These features basically include the **scope(visibility) and life-time which help us to trace the existence of a particular variable during the runtime of a program**. The following storage classes are most often used in C programming. Storage classes in C are used to determine the **lifetime, visibility, memory location, and initial value of a variable.**



Automatic Variables

A variable declared inside a function without any storage class specification is by default an automatic variable. They are created when a function is called and are destroyed automatically when the function execution is completed. **Automatic variables can also be called local variables** because they **are local to a function.** By default, they are assigned to **undefined values.**

Coding Example_1:

```
#include<stdio.h>
 int main()
{       int i=90;          // by default auto because defined  inside a function
        auto float j=67.5;
        printf("%d %f\n",i,j);
}
```

```
int f1()
{
        int a;  // by default auto because defined  inside a function f1()
                // Not accessible outside this function.
}
```

External variables

The extern keyword is used before a variable **to inform the compiler that the variable is declared somewhere else**. The extern declaration does not allocate storage for variables. All functions are of type extern. **The default initial value of external integral type is 0 otherwise NULL. We can only initialize the extern variable globally, i.e., we cannot initialize the external variable within any block or method.**

Coding Example_2: Demo of extern

```
int main()
{
        extern int i;     // Information saying declaration exist somewhere else and make it available during linking
                printf("%d\n",i);   // if u comment this line, it throws an error: i undeclared
}
int i=23;
```

Note: An external variable can be declared many times but can be initialized at only once. The extern modifier is most commonly used when there are two or more files sharing the same global variables or functions.

Coding Example_3: Two files share the same global variable

Sample.c

```
#include <stdio.h>
 int count ;

extern void write_extern();
```

```
int main() {
        count = 5;
        write_extern();
        return 0;
}
```

Sample1.c

```
#include <stdio.h>
 extern int count;
 void write_extern(void)
{
  printf("count is %d\n", count);
}
```

Here, extern is being used to declare count in the second file, where as it has its definition in the first file, main.c.

```
// please check the execution steps below
gcc Sample.c Sample1.c –c
gcc Sample.o Sample1.o
```

Output - count is 5

Static variables

A static variable tells the compiler to persist the variable until the end of program. Instead of creating and destroying a variable every time when it comes into and goes out of scope, static is initialized only once and remains into existence till the end of program. A static variable can either be local or global depending upon the place of declaration.

Scope of local static variable remains inside the function in which it is defined but the life time of local static variable is throughout that program file. Global static variables remain restricted to scope of file in each they are declared and life time is also restricted to that file only. **All static variables are assigned 0 (zero) as default value.**

Coding Example_4: Demo for Life time of Static variable

```
#include<stdio.h>
int* f1();
```

```
int main()
{
        int *res = f1();
        printf("Res is %p %d\n",res,*res);     //same address in all three outputs. Then 11
        res = f1();
        printf("Res is %p %d\n",res,*res);     // 12
        res  =  f1();
        printf("Res is %p %d\n",res,*res);     // 13
        return 0;
}
in t* f1()
{
        static int a= 10;// memory is shared. This line is ignored after first function call
        a++ ;
        return &a;     // valid because life time of static variable is throughout the file execution
}
```

Coding Example_5: Understand the differences between local and global static variable.

```
#include<stdio.h>
void f1();
static int j=20;              // global static variable: cannot be used outside this program file
int k=23;                     //global variable: can be used anywhere by linking with this file.
                                      //By default   has extern with it.
int main( )
{
        f1();                 // 0  20
        f1();                 // 0  21
        f1();                 // 0  22
        return 0;
}
void f1()
{       int i=0;     printf("%d %d\n",i,j);        i++;     j++;     }
```

Register variables

Registers are faster than memory to access, so the variables which are most frequently used in a C program can be put in registers using register keyword. The keyword register hints to compiler that a given variable can be put in a register. It's compiler's choice to put it in a register or not.

Register is an integral part of the processor. It is a very fast memory of computer mainly used to execute the programs and other main operation quite efficiently. They are used to quickly store, accept, transfer, and operate on data based on the instructions that will be immediately used by the CPU. Main operations are fetch, decode and execute. Numerous fast multiple ported memory cells are the atomic part of any register. Generally, compilers themselves do optimizations and put the variables in register. If a free register is not available, these are then stored in the memory only. If & operator is used with a register variable then compiler may give an error or warning (depending upon the compiler used), because when a variable is a register, it may be stored in a register instead of memory and accessing address of a register is invalid. **The access time of the register variables is faster than the automatic variables.**

Mostly used cpu registers are Accumulator, Flag Register, Address Register (AR), Data Register (DR), Program Counter (PC), Instruction Register (IR), Stack Control Register (SCR), Memory Buffer Register (MBR) and Index register (IR).

Coding Example_6:

```
#include<stdio.h>
int main()
{
        register int i = 10;
        int* a = &i;  // error
        printf("%d", *a);     getchar();
        return 0;
}
```

Coding Example_7: We can store pointers into the register, i.e., a register can store the address of a variable.

```
#include<stdio.h>
int main()
{
        int i = 10;
        register int* a = &i;    // fine
        printf("%d", *a);     getchar();
        return 0;
}
```

Note: Static variables cannot be stored into the register since we cannot use more than one storage specifier for the same variable.

Global variables

The variables declared outside any function are called global variables. They are not limited to any function. **Any function can access and modify global variables**. Global variables are **automatically initialized to 0 at the time of declaration.**

Coding Example_8:

```
#include<stdio.h>
int i = 100;      int j;
int main()
{       printf("%d\n",i);                    // 100
        i=90;          printf("%d\n",i);     // 90
        f1();          printf("%d\n",i);     // 40
        printf("%d\n",j);                    // 0 by default, global variable is 0 if just declared
        return 0;
}

int f1()
{                      i = 40;       // Any function can modify the global variable.
}
```

Dive into Storage Classes and keep Coding!!

Department of Computer Science and Engineering,
PES University, Bangalore, India

Lecture Notes
Problem Solving With C
UE24CS151B

Lecture #11
Callback in C

By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU

Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)

Unit #: 2

Unit Name: Counting, Searching and Sorting

Topic: Callback in C

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

In computer programming, a **callback** is also known as a "**call-after function**". The callback execution may be immediate or it might happen at a later point in time. Programming languages support call backs in different ways, often implementing them with subroutines, lambda expressions, blocks, or function pointers. In C, a **callback function is a function that is called through a function pointer/pointer to a function.** A callback function has a specific action which is bound to a specific circumstance. It is an important element of GUI in C programming.

Let us understand the use of **pointer to a function or a function pointer.**

Function Pointer points to code, not data. The function pointer stores the start of executable code. The function pointers points to functions and store its address.

Syntax: int (*ptrFunc) ( );

The **ptrFunc** is a pointer to a function that takes no arguments and returns an integer. If parentheses are not given around a function pointer then the compiler will assume that theptrFunc is a normal function name, which takes nothing and returns a pointer to an integer. Function pointers are not used for allocating or deallocating memory, instead used in reducing code redundancy.

Consider the below cases:

Case 1: int *a1(int, int, int); // a1 is a function which takes three int arguments and returns a pointer to int.

Case 2: int (*p)(int, int, int); //p is a pointer to a function which takes three int as parameters and returns an int.

We can assign a function name to p as long as the function has the same signature which means that the function takes three integer parameters and returns an integer. **The function name acts like a pointer to a function.**

```
int a2(int,int,int);
p = a2;
int y = p(2,3,4); // This statement is the same as calling a2.
```

Why is the callback function required?

Let us answer a few questions to understand the importance of callback function.

How to write a generic library for sorting algorithms such as bubble sort, shell short, shake sort, quick sort?

How to create a notification event to set a timer in your application?

How to have one common method to develop libraries and event handlers for many programming languages?

How to redirect page action is performed for the user based on one click action?

How to extend the features of one function using another one?

A callback is any executable code that is passed as an argument to other code, which is used to call (execute) the argument at a given time. In simple language, if a function name is passed to another function as an argument to call it, then it will be called as a Callback function.

Coding Example_1:

```
int what(int x, int y, int z, int (*op)(int, int,int))
{
        return op(x, y, z);
}
```

Observe the third argument op. It is a pointer to a function which takes three int arguments and returns an int. The return statement in turn calls the function with three arguments. This function does not know which function is getting called. It depends on the value of op. **Compiler knows the type and not the value. The value of a variable is a runtime mechanism.**

```
int add(int x, int y, int z)
{       return x + y + z;   }
int mul(int x, int y, int z)
{       return x * y * z;   }
int main()
{       int (*p)(int, int, int);
```

```
    p = add; // p = &add; Both are equivalent
    int res = p(2,3,4); // (*p)(2, 3, 4); //Both are equivalent
    printf("result is %d\n", res);
    p = mul;
    res = p(2,3,4);
    printf("result is %d\n", res);
    // function name can be directly passed as an argument to function with callback
    printf("sum is %d\n", what(1, 3, 4, add));
    printf("product is %d\n", what(5, 3, 4, mul));
    return 0;
}
```

Coding Example_2: If you are aware of Python's Functional programming constructs such as map, reduce and filter, we will be implementing those here in C using callback functions.

// Mimic map, filter and reduce function of python

```
#include<stdio.h>
void mymap(int *a,int *b,int n,int (*p)(int));
void disp(int *a, int n);
int myfilter(int *a,int *b,int n,int (*p)(int));
int myreduce(int *a, int n, int (*op)(int, int));

int incr(int x)
{       return x+1;     }
int is_even(int x)
{       return x%2 == 0;        }
int is_greater_than_22(int x)
{       return x > 22;          }
int add(int x,int y)
{       return x+y;             }
```

```
int main()
{       int a[] = {11,22,33,44,55};
        int n = sizeof(a)/sizeof(*a);
        int b[100];
        printf("a is: \n");         disp(a,n);
        mymap(a,b,n,incr);
        printf("\nAfter incrementing using map, b is: \n");    disp(b,n);

        int c = myfilter(a,b,n,is_even);
        printf("\nAfter filtering only even, b is: \n"); disp(b,c);

        c = myfilter(a,b,n,is_greater_than_22);
        printf("\nAfter filtering only number greater than 22, b is:\n");        disp(b,c);

        int result = myreduce(a,n,add);
        printf("\nSum of all the elements using reduce: %d\n",result);
        return 0;
}
void mymap(int *a,int *b,int n,int (*p)(int))
{       for(int i = 0;i<n;i++)              {       b[i] = (*p)(a[i]);        }
}

int myfilter(int *a,int *b,int n,int (*p)(int))
{
        int count = 0;
        for(int i = 0;i<n;i++)     {
                if (p(a[i]))
                {       b[count] = a[i];          count++;}
        }
        return count;
}
```

```
void disp(int *a, int n)
{
        for(int i = 0;i<n;i++)   {                  printf("%d\t", a[i]);              }
}
int myreduce(int *a, int n, int (*op)(int, int))
{
        int res = 0;
        for(int i = 0;i<n;i++)
        {                   res = op(res,a[i]);       }
        return res;
}
```

```
C:\Users\Dell>a
a is:
11      22      33      44      55
After incrementing using map, b is:
12      23      34      45      56
After filtering only even, b is:
22      44
After filtering only number greater than 22, b is:
33      44      55
Sum of all the elements using reduce: 165
```

Happy Coding using Callback!!

Department of Computer Science and Engineering,
PES University, Bangalore, India

Lecture Notes
Problem Solving With C
UE24CS151B

Lecture #12
Recursion

By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU

Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)

Unit #: 2

Unit Name: Counting, Searching and Sorting

Topic: Recursion

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors.

Course outcomes: At the end of the course, the student will be able to:

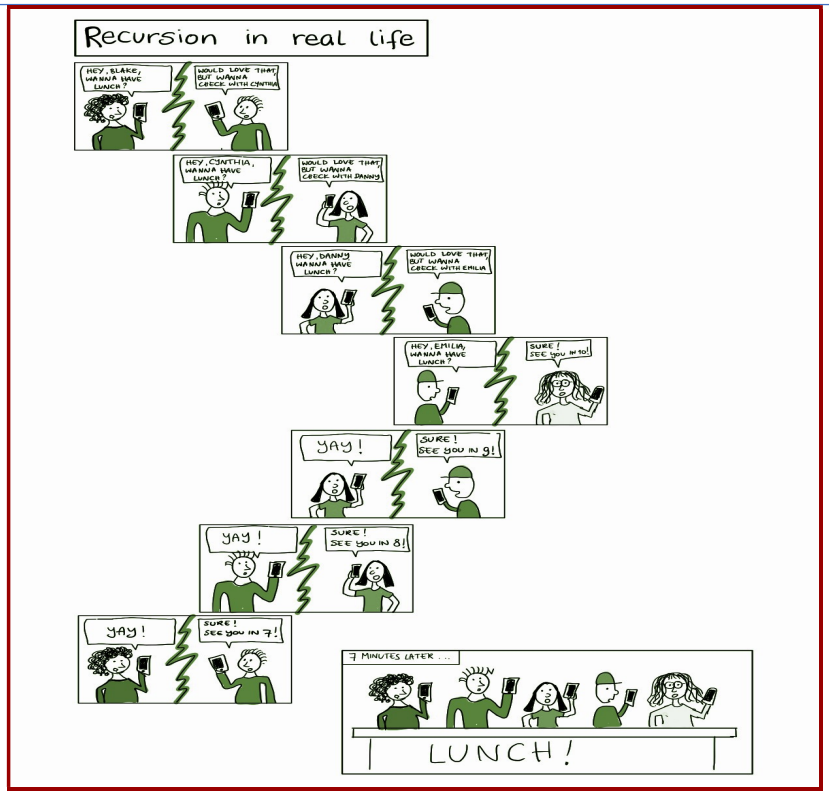
- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

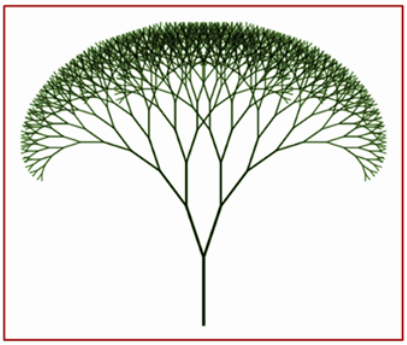
Sindhu R Pai

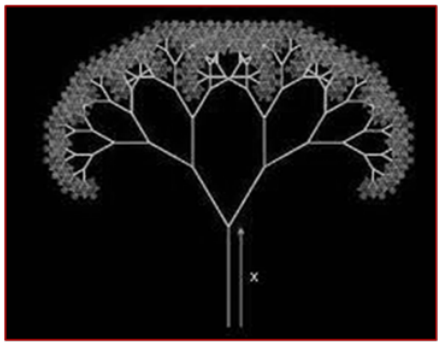
Theory Anchor, Feb - May, 2025

Dept. of CSE,

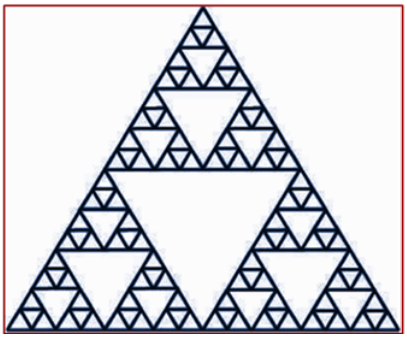
PES University

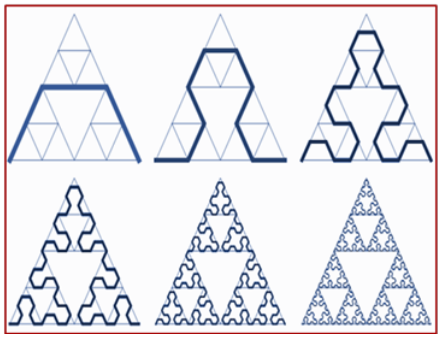






Fractals





The Sierpinski Triangle

Introduction

A function may call itself directly or indirectly. **The function calling itself with the termination/stop/base condition is known as recursion**. Recursion is used to solve various problems by dividing it into smaller problems. We can **opt if and recursion instead of looping statements.** In recursion, we try to **express the solution to a problem in terms of the problem itself, but of a smaller size.**

Key Characteristics:

Base Case – A condition that stops the recursion to prevent infinite recursion.

Recursive Case – The function calls itself with a smaller or simpler input.

Coding Example_1:

```
int main()
{
        printf("Hello everyone\n");
        main();
        return 0;
 }
```

Execution starts from main() function. 'Hello everyone' gets printed. Then call to main() function. Again, 'Hello everyone' gets printed and these repeats. We have **not defined any condition for the program to exit, results in Infinite Recursion.** In order **to prevent infinite recursive calls, we need to define proper base condition in a recursive function.**

Common Applications:

- Factorial Calculation
- Fibonacci Sequence
- Tree Traversals (Binary Trees)
- Graph Traversal (DFS)
- Backtracking Problems (Sudoku Solver, N-Queens)
- Sorting Algorithms (Merge Sort, Quick Sort)

Let us write Iterative and Recursive functions to find the Factorial of a given number.

Coding Example_2:

```
int fact(int n);
int main()
{       int n = 6;
        printf("factorial of %d is %d\n",fact(n));
        return 0;
}
```

Iterative Implementation:

```
int fact(int n)
{       int result = 1;
      int i;
        for (i = 1; i<=n; i++)
      { result = result * i;    }
  return result;
}
```

Recursive Implementation:

Logic: If n is 0 or n is 1, result is 1 Else, result is n*(n-1)!

```
int fact(int n)
{       if (n==0)
                return 1;
        else
        {               return n*fact(n-1);             }
}
```

Pictorial representation:

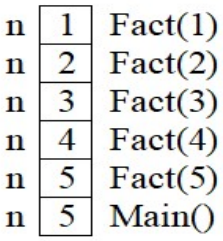
=factorial(5)
=5*factorial(4)
=5*4*factorial(3)
=5*4*3*factorial(2)
=5*4*3*2*factorial(1)
=5*4*3*2*1*factorial(0)
=120

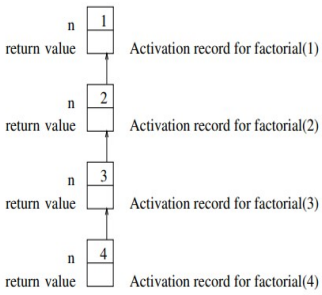
When n is 6

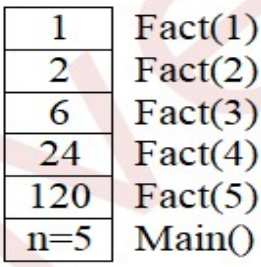
```
factorial of 6 is 720


...Program finished with exit code 0
Press ENTER to exit console.
```

Activation Record

- Each function call generates an instance of that function containing memory for each parameter, memory for each local variable and memory for return values. This chunk of memory is called as an **activation record**.
- To support recursive function calls, the run-time system treats memory as a stack of activation record.
- Computing the factorial (n) requires allocation of 'n' activation records on the stack.







Coding Example_3: Printing the binary representation of a give number

```
#include<stdio.h>
int what1(int n)
{
        if(n==0) return 0;
        else return (n%2)+10*what1(n/2);
}
int main()
{       printf("%d",what1(8));          return 0;     }
```

```
1000

...Program finished with exit code 0
Press ENTER to exit console.
```

What does the below code do? Write the activation record for each.

```
#include<stdio.h>
int what2_v1(int n);
int what2_v2(int n);
int main()
{       int n = 10;
        printf("%d",what2_v1(n));
        printf("%d",what2_v2(n));
        return 0;
}
```

Finds the count of 1s in the binary representation of the number.

```
2

...Program finished with exit code 0
Press ENTER to exit console.
```

```
int what2_v2(int n)
{
        if(!n) return 0;
        else if (!(n%2)) return what2_v2(n/2);
        else{    return 1+what2_v2(n/2);   }
}
int what2_v1(int n)
{
        if(n==0) return 0;
        else return (n&1)+ what2_v1(n>>1);
}
```

Coding Example_4: Finds a to the power b

```
#include <stdio.h>
int what3_v1(int,int);
int main()
{       int a=5,b=2;
        printf("%d to the power %d is  %d\n",a,b,what3_v1(a,b));
        //printf("%d to the power %d is  %d\n",a,b,what3_v2(a,b));
        return 0;    }
int what3_v1(int b,int p)
{     int result=1;
       if(p==0)
          return result;
       else
          result=b*(what3_v1(b,p-1));       }
int what3_v2(int b, int p){
        if(!p)
           return 1;
        else if(!(p % 2))
           return what3_v2(b*b, p/2);
        else
           return b*what3_v2(b*b,p/2); }
```

```
5 to the power 2 is  25


...Program finished with exit code 0
Press ENTER to exit console.
```

Happy Coding using Recursion!!

Department of Computer Science and Engineering,

PES University, Bangalore, India

Lecture Notes
Problem Solving With C
UE24CS151B

Lecture #13
Problem Solving using Recursion

By,

Prof. Sindhu R Pai,

Theory Anchor, Feb-May, 2025

Assistant Professor

Dept. of CSE, PESU

Many Thanks to

Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)

Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)

Unit #: 2

Unit Name: Counting, Searching and Sorting

Topic: Problem Solving using Recursion

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Solve the below programs using Recursion

Link for Solution: https://drive.google.com/file/d/1CZKirLd2E1j7leLG_aEPsPLca5jIjXV-/view?usp=drive_link

Level-1:Banana

1. A young programmer is learning recursion. His mentor gives him a challenge: Can you **add two integers without using the + operator directly**? Instead of normal addition, he must use recursion by incrementing one number while decrementing the other until one of them becomes zero.
   Input: 4 3 **Expected Output:** 7

2. Fibonacci numbers appear in nature, such as the arrangement of leaves, flowers, and shells. Develop an application for biologists to analyze these patterns using a recursive function Fibonacci(n) to generate the n^{th} Fibonacci number. Display appropriate message for the negative value of n and 0.
   Input: n = 6 **Expected Output:** 5
   Explanation:
   0,1,1,2,3,5,8,13,21,34... Fibonacci sequence is a sequence in which each number is the sum of the two preceding numbers. First two numbers are fixed in the series.

3. A bank's security team discovered that fraudsters were tampering with account numbers. To prevent this, they introduced a checksum verification system. Each account number must pass a test where the sum of its digits is calculated to ensure authenticity. Can you help the bank by writing a recursive function to compute the sum of digits in an account number?
   Input: 1234 **Expected Output:** Sum of digits: 10
   Input: -1234 **Expected Output:** Sum of digits: 10

4. In a secret agency, messages including positive integers are encrypted using a **special formula: a^b**. The spies need to quickly **compute large powers** to send secure codes. Using recursion, they can break down the calculation into smaller steps, making it faster and more efficient. Implement a recursive function to help them encrypt their messages?
   Input: a = 2, b = 5
   Expected Output: 2^5 = 32
   Input: a = 0, b = 0
   Expected Output: 0^0 = 1
   Input: a = 0, b = 1
   Expected Output: 0^1 = 0
   Input: a = 1, b = 0
   Expected Output: 1^0 = 1

5. In an ancient civilization, architects used a special formula on positive integers to calculate the correct proportions of their buildings. They believed **that using the Greatest Common Divisor (GCD) of two measurements ensured structural stability.** As an engineer studying these historical techniques, you need to write a recursive function **to compute the GCD of two given numbers. Display appropriate message for negative integers.**
   Input: a = 56, b = 98
   Expected Output: GCD of 56 and 98 is 14

Level-2:Orange

6. NASA scientists are working on decoding signals from an alien spacecraft. **The signals are sent in binary format, and they need to be converted to decimal numbers to understand the coordinates of the spacecraft**. Since binary numbers can be of any length, recursion is a natural choice for conversion.
   Input: 1011
   Expected Output: Decimal equivalent: 11

7. One day, Rahul went to an ATM to deposit money. Due to system error, the ATM processed his transaction in reverse order. Now, the bank's software needs **to reverse the digits** of the transaction ID to verify the correct transaction. Since numbers can have varying lengths, recursion is an efficient way to solve this problem without using strings.
   Input: 1234
   Expected Output: Reversed number: 4321

8. In an ancient temple, scholars discovered a mysterious set of numbers carved into a stone. They believed these numbers were **magical because they read the same forward and backward.** To decode their meaning, they requested the team to build an application to **check if a number is a palindrome or not.** Since numbers can be of varying lengths, recursion is the best way to solve this problem.
   Input: 1221
   Expected Output: 1221 is a palindrome

9. A digital wallet app needs a fraud detection system that checks if a positive transaction ID is too large**. To simplify the ID, they sum up all its digits recursively until they get a single-digit number.** This is similar to how some banking checksum algorithms work. Handle negative inputs with a proper message.
   Input: 9875
   Expected Output: Single digit id is 2
   Explanation:
   9 + 8 + 7 + 5 = 29
   2 + 9 = 11
   1 + 1 = 2 (Final output)

Level-3:Jackfruit

10. A mathematician named Aryan discovered an ancient manuscript containing a numerical pattern known as **Pascal's Triangle.** The legend says that a hidden treasure can be unlocked if he deciphers the number at a specific position in the triangle.

    According to the manuscript, the **0th row** starts with **1**, and each number in the next row is formed by adding the two numbers directly above it. Aryan needs to find the element at a given row **N** and column **M** to move forward in his quest. However, the calculations become tricky for large values, and he decides to use recursion to solve the problem efficiently.

    Can you help Aryan find the treasure by writing a recursive function that determines the number at the **Nth row and Mth column** of Pascal's Triangle? Handle negative values accordingly with a proper message.
    Pascal's Triangle

```
                1
              1   1
            1   2   1
          1   3   3   1
        1   4   6   4   1
      1   5   10  10  5   1
    1   6   15  20  15  6   1
  1   7   21  35  35  21  7   1
```

    Input: 4 2
    Expected Output: Pascal's Triangle value at (4, 2) is 6
    Explanation: The 4th row of Pascal's Triangle is [1, 4, 6, 4, 1].
    The element at the 2nd column (0-based index) in the 4th row is 6.

11. A monk in a temple follows an ancient tradition where he **moves a stack of 3 disks from one rod to another using a helper rod.** The rule is that only **one disk can be moved at a time, and no larger disk can be placed on a smaller one.** This is a perfect example of recursion because solving it for n disks depends on solving it for n-1 disks. Implement the function hanoi to do the requirement.
    Input: 3
    Client code: int n = 3;
    hanoi(n, 'A', 'C', 'B');
    Function declaration: void hanoi (int n, char from, char to, char aux);
    Expected Output:
    Move disk 1 from A to C
    Move disk 2 from A to B
    Move disk 1 from C to B
    Move disk 3 from A to C
    Move disk 1 from B to A
    Move disk 2 from B to C
    Move disk 1 from A to C

12. A mobile app developer is working on a numeric password generator. The challenge is **to generate all possible 3-digit numbers using the digits available on a mobile keypad (0-9).** Since there are 10 digits (0-9), the system should recursively form all unique 3-digit combinations where each digit can be used multiple times.
    Expected output:
    000
    001
    002
    ...
    997
    998
    999

Happy Coding using Recursion!!

Department of Computer Science and Engineering,
PES University, Bangalore, India

Lecture Notes
Problem Solving With C
UE24CS151B

Lecture #14
Searching using C

By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU

Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)

Unit #: 2

Unit Name: Counting, Searching and Sorting

Topic: Searching using C

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

Even a single day does not pass without we searching for something in our day to day life. Might be car keys, books, pen, mobile charger and what not. Same is the life of a computer. There is so much data stored in it and whenever user asks for some data, computer has to search it's memory to look for the data and make it available to the user. And the computer has it's own techniques to search through it's memory in a faster way.

Why Searching?

We often need to find one particular item of data amongst many hundreds, thousands, millions or more. For example, if one wants to find someone's phone number in your phone, or a particular business's address from address book.

A searching algorithm can be used to find the data without spending much time. What if you want to write a program to search a given integer in an array of integers? Two popular algorithms available: **Linear Search and Binary Search**

Linear Search

A linear search, also known as a **sequential search**, is a method of finding an element within a collection. It checks each element of the list sequentially until a match is found or the whole list has been searched. It **returns the position of the element in the array, else it return -1**. Linear Search can be applied on any unordered or ordered collection of elements.

Coding Example_1: Implementation of Linear Search

```
#include<stdio.h>
int linear_search(int *a, int n, int key);
int main()
{
        int key;
        int a[] = {22,55,77,99,25,90,100};
        int n = sizeof(a)/sizeof(a[0]);
        printf("Enter the element to be searched in the given collection\n");
        scanf("%d", &key);
```

```
        int pos = linear_search(a, n, key);
        if(pos != -1)
                printf("Element is found at %d\n", pos);
        else
                printf("Element is not found in the collection\n");
        return 0;
}
int linear_search(int *a, int n, int key)
{
        int i; int found = 0; int pos = -1;
        for(i = 0; i< n; i++)
        {
                if(a[i]==key && found == 0)
                {
                        found = 1;
                        pos = i;
                }
        }
        return pos;
}
```

```
C:\Users\Dell>a
Enter the element to be searched in the given collection
22
Element is found at 0

C:\Users\Dell>a
Enter the element to be searched in the given collection
56
Element is not found in the collection

C:\Users\Dell>a
Enter the element to be searched in the given collection
100
Element is found at 6
```

Binary Search

Binary Search is used to **search an element in a SORTED ARRAY**. The following steps are applied to search an element.

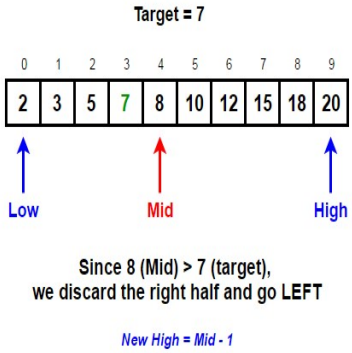
1. Start by comparing the element to be searched with the **element in the middle of the array.**
2. If we **get a match, we return the index** of the middle element.
3. If we do not get a match, **we check whether the element to be searched is less or greater than in value than the middle element.**
4. If the element/number to be searched is greater in value than the middle number, then we pick the elements on the right side of the middle element(as the array is sorted, hence on the right, we will have all the numbers greater than the middle number), and

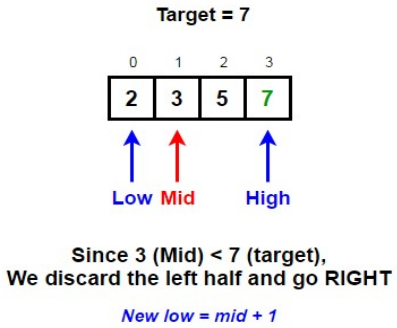
start again from the step 1.

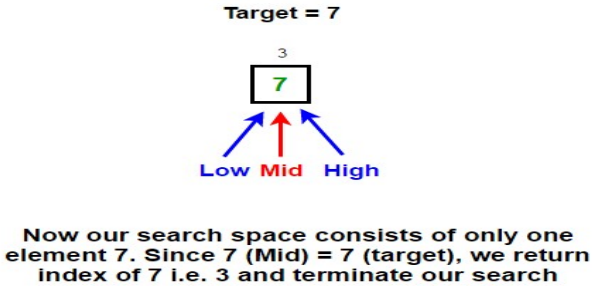
5. If the element/number to be searched is lesser in value than the middle number, then we pick the elements on the left side of the middle element, and start again from the step 1.

Binary Search Representation

Example: Take an input array by name arr = [2, 3, 5, 7, 8, 10, 12, 15, 18, 20] and **target or key to be searched is 7.**







Note: Between 2,3,5,7 and 7, write diagrams for missing two iterations. Drawing these two iterations might help you to understand better.

Binary Search Algorithm (Iterative solution)

Algorithm Binary_Search(list, item)

1. Set L to 0 and R to n: 1
2. if L > R, then Binary_Search terminates as unsuccessful
3. else,Set m (the position in the mid element) to the floor of (L + R) / 2
4. if A_m < T, set L to m + 1 and go to step 2
5. if A_m > T, set R to m: 1 and go to step 2
6. Now, A_m == T, the search is done and return (m).

Explanation

The iterative binary search algorithm works on the principle of "**Divide and Conquer**". First it divides the large array into smaller sub-arrays and then finding the solution for one sub-array. It finds the position of a key or target value within an array and compares the target value to the middle element of the array, if they are unequal, the half in which the target cannot lie is eliminated. The search continues on the remaining half until it is successful.

Binary Search Algorithm (Recursive solution)

BinarySearch.(x:target;{a1,a2,…,an}:sorted list of items;

1. begin:pos;end:pos
2. mid:=(begin+end)/2
3. If begin>end:return -1
4. else if x==amid:return mid
5. else if x<amid:return BinarySearch(x;{a1,a2,…,an};begin;mid-1)
6. else return BinarySearch(x;{a1,a2,…,an};mid+1,end);

Explanation

In recursive approach, the function BinarySerach is recursively called to search the element in the array. It accepts the input from the user and store it in **m.** The array elements are declared and initialized.In the recursive call to function **the array, the lower index, higher index and the number are passed as arguments** to be searched again and again until element is found. If not found, then it returns -1.

Coding Example_2: Implementation of Binary Search (iterative & recursive) on an array of integers

```
#include<stdio.h>
int mysearch(int a[],int low,int high,int key)
{
        /* Binary search - Iterative Implementation
        int pos = -1;
        int found = 0;  // if found variable is not created, what is the problem. Think about it? Is there any other way?
```

```
while(low<=high && found ==0)
//while(low<=high && pos != mid)
// while(low<=high && pos == -1)
{
        int mid = (low+high)/2;
        if(a[mid]==key)
        { pos = mid; found = 1;  }
        else if(key<a[mid])
                high = mid-1;
        else
                low = mid+1;
}
return pos;
*/
```

```
C:\Users\Dell>a
Enter the element to be searched in the given collection
100
not found
C:\Users\Dell>a
Enter the element to be searched in the given collection
21
not found
C:\Users\Dell>a
Enter the element to be searched in the given collection
1
not found
C:\Users\Dell>a
Enter the element to be searched in the given collection
7
found at 3
```

```
// Binary Search - Recursive Implementation
if(low > high)  // base condition
        return -1;
else
{
        int mid = (low+high)/2;
        if(a[mid]==key)
        {       return mid;     }
        else if(key<a[mid])
                return mysearch(a,low,mid-1,key);
        else
                return mysearch(a,mid+1,high,key);
}
}
```

// client code is as below

```
int main()
{       int key;
        int a[] =  {2, 3, 5, 7, 8, 10, 12, 15, 18, 20};
        int n = sizeof(a)/sizeof(a[0]);
        printf("Enter the element to be searched in the given collection\n");
        scanf("%d", &key);
        int res = mysearch(a,0,n-1,key);
        if(res == -1)               printf("not found");
        else                printf("found at %d\n",res);
        return 0;
}
```

Happy Searching using C!!

Department of Computer Science and Engineering,
PES University, Bangalore, India

Lecture Notes
Problem Solving With C
UE24CS151B

Lecture #15
Sorting using C

By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU

Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)

Unit #: 2

Unit Name: Counting, Searching and Sorting

Topic: Sorting using C

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

There are so many things in our real life that we need to search for, like a particular record in the database, roll numbers in the merit list, a particular telephone number in a telephone directory, a particular page in a book etc. The concept of sorting came into existence, making it easier for everyone to arrange data in order to make sure that the searching process is easy and efficient. **Arranging the data in ascending or descending order is known as sorting.**

Why Sorting?

Think about searching for something in a sorted drawer and unsorted drawer. If the large data set is sorted based on any of the fields, then it becomes easy to search for a particular data in that set. Sometimes in real world applications, it is necessary to arrange the data in a sorted order to make the searching process easy and efficient.

Example: Candidates selection process for an Interview- Arranging candidates for the interview involves sorting process (sorting based on qualification, Experience, skills or age etc.

Sorting Algorithms

Sorting algorithm **specifies the way to arrange data in a particular order**. Most common orders are in numerical or lexicographical order. **Depending on the data set, Sorting algorithms exhibit different time and space complexity.**

Following are samples of sorting algorithms.

1. Bubble Sort
2. Insertion Sort
3. Quick Sort
4. Merge Sort
5. Radix Sort
6. **Selection Sort -- Dealt in detail**
7. Heap Sort

Selection sort

A simple sorting algorithm that works by **repeatedly selecting the smallest (or largest) element from the unsorted portion of the array and swapping it with the first unsorted element.** It follows an **in-place** and **comparison-based** approach. The algorithm **maintains two subarrays in a given array.**

1) The subarray which is already sorted.

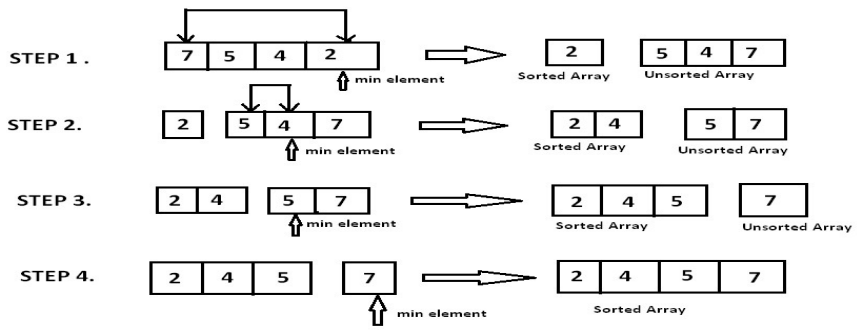
2) Remaining subarray which is unsorted.

The sorted part of the array is empty and unsorted part is the given array. Sorted part is placed at the left, while the unsorted part is placed at the right. If it is used to arrange data in an ascending order, the smallest number is selected and placed at the beginning of the collection. When the next pass takes place the second minimum number is selected and placed in the second position. The process is repeated until all the elements are placed in the correct order. With each pass the elements are reduced by 1 as one element is sorted and placed in a position.

Algorithm (Step-by-Step)

1. Start with the first element as the minimum.
2. Traverse through the remaining unsorted elements to find the actual minimum.
3. Swap the minimum found with the first element of the unsorted part.
4. Move the boundary of the sorted region by one position to the right.
5. Repeat until the entire array is sorted.

Pictorial Representation



Coding Example_1: Selection sort on an Array of integers

```
#include<stdio.h>
void sort(int *a, int n);
int main()
{       int a[ ] = {22,19,14,25,99,81};
        int n = sizeof(a)/sizeof(*a);
        printf("before sorting\n");
        for(int i = 0; i < n; i++)
        {               printf("%d\t",a[i]);            }
        sort(a,n);
        printf("\nafter sorting\n");
        for(int i = 0; i < n; i++)
        {               printf("%d\t",a[i]);            }
        return 0;
}
void sort(int *a, int n)
{
        int pos;
        for(int i = 0; i< n-1;i++)
        {
                pos = i;
                for(int j = i+1; j<=n-1; j++)   // or j < n
                {
                        if(a[pos] > a[j])
                        {       pos = j;                }
                }
                if(pos != i)
                {
                        int t = a[pos];  a[pos] = a[i];   a[i] = t;   //swap(&a[pos], &a[i]);
                }
        }
}
```

Coding Exampe_2: Sort the elements of the array given by the user

```
#include<stdio.h>
void sort(int *a, int n);
void swap(int *x, int *y);
void disp(int *a, int n);
void read(int *a, int n);
int main()
{               int a[100];             int n;
                printf("enter the number of elements u want to sort\n");
                scanf("%d",&n);
                printf("enter %d elements\n",n);        read(a,n);
                printf("before sorting\n");             disp(a,n);
                sort(a,n);
                printf("after sorting\n");              disp(a,n);
                return 0;
}

void disp(int *a,int n)
{       for(int i = 0;i<n;i++)
        {   printf("%d ",a[i]);       }
        printf("\n");
}

void read(int a[],int n)
{       for(int i = 0;i<n;i++)
        {               scanf("%d",&a[i]);    }
}

void swap(int *x,int *y)
{       int temp = *x;          *x = *y;        *y = temp;    }
```

```
void sort(int *a, int n)
{
        int i,pos,j;
        for(i = 0;i<n-1;i++)
        {
                pos = i;
                for(j = i+1;j<n;j++)
                {
                        if(a[pos]>a[j])
                                pos = j;
                }
                if(pos != i)     // think about this – Is it really required?
                        swap(&a[pos],&a[i]);
        }
}
```

Few points for you to think!

- How would the algorithm look if implemented recursively?
- Not suitable forlarge datasets. Then, where might Selection Sort still be useful?
- Can we reduce the number of comparisons in Selection Sort?
- It doesn't optimize for already sorted lists. Can it be modified to detect an already sorted list early?

Happy Sorting using C!!

Department of Computer Science and Engineering,
PES University, Bangalore, India

Lecture Notes
Problem Solving With C
UE24CS151B

Lecture #16
Problem Solving using Callback
[Counting, Searching and Sorting]

By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU

Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)

Unit #: 2

Unit Name: Counting, Searching and Sorting

Topic: Problem Solving using Callback [Counting, Searching and Sorting]

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

Counting, sorting, and searching are fundamental operations in computer science. **Counting** involves tracking occurrences or frequencies, such as counting the appearances in a collection of elements. **Sorting** arranges data in a specific order (ascending or descending) using algorithms like Bubble Sort, **Selection sort,** Merge Sort, or Quick Sort, improving efficiency in later operations. **Searching** finds specific elements within data using methods like Linear Search (checking each item sequentially) or Binary Search (dividing sorted data for faster lookup). These **operations are essential for data processing, optimization, and efficient retrieval in real-world applications like databases, search engines, and analytics**.

Counting

In C, counting elements in an array based on specific conditions can be efficiently handled using callback function. This **method allows us to apply different conditions dynamically without modifying the counting function** itself.

Coding Example_1: Counting the number of positive and negative elements, even and odd elements using a generic function.

```
#include <stdio.h>
int (*ConditionFunc)(int);
// Function to count elements in an array based on a callback condition
int countElements(int *arr, int size, int (*ConditionFunc)(int)) {
   int count = 0;
   for (int i = 0; i < size; i++) {
      if (ConditionFunc(arr[i])) {  // Apply the callback condition
         count++;       }
   }
   return count;
}
// Callback function: Check if a number is even
int isEven(int num) {     return num % 2 == 0;        }
// Callback function: Check if a number is odd
int isOdd(int num) {    return num % 2 != 0;          }
```

```
// Callback function: Check if a number is greater than a given threshold
int isGreaterThanFive(int num) {    return num > 5;         }
// Callback function: Check if a number is negative
int isNegative(int num) {    return num < 0;              }

int main() {
    int numbers[] = {1, -3, 4, 6, -8, 9, 11, 15, 2, -7};
    int size = sizeof(numbers) / sizeof(numbers[0]);
    // Using different callbacks to count based on conditions and display the counts
    int evenCount = countElements(numbers, size, isEven);
    int oddCount = countElements(numbers, size, isOdd);
    int greaterCount = countElements(numbers, size, isGreaterThanFive);
    int negativeCount = countElements(numbers, size, isNegative);
    printf("Count of even numbers: %d\n", evenCount);
    printf("Count of odd numbers: %d\n", oddCount);
    printf("Count of numbers greater than 5: %d\n", greaterCount);
    printf("Count of negative numbers: %d\n", negativeCount);
    return 0;
}
```

```
C:\Users\Dell>a
Count of even numbers: 4
Count of odd numbers: 6
Count of numbers greater than 5: 4
Count of negative numbers: 3
```

Sorting

Sorting is a fundamental operation in problem-solving and algorithm design. **Different scenarios require different sorting orders—ascending, descending, or even custom sorting based on specific criteria**. Instead of writing separate sorting logic for each case, callback functions provide a powerful way to make sorting flexible and reusable. For example, in Selection Sort, we **can use a callback function to compare two elements dynamically.** If sorting in **ascending order, the callback ensures smaller elements come first**; if sorting in **descending order, it ensures larger elements come first**. By leveraging callback functions, we can solve a variety of sorting-related problems efficiently, making the code more adaptable and scalable. This approach is **especially useful in real-world applications where sorting criteria may change based on user preferences, data structures, or optimization requirements.**

Coding Example_2: Sorting a collection of integers in ascending and descending order using callback mechanism

```
#include<stdio.h>
// Callback function for ascending order
int ascending(int a, int b) {
   return a > b; // Swap if a is greater than b
}

// Callback function for descending order
int descending(int a, int b) {
   return a < b; // Swap if a is smaller than b
}

//Sort using Callback
void sort(int *a, int n, int (*compare)(int, int))
{
        int pos;
        for(int i = 0; i< n-1;i++)  {
                pos = i;
                for(int j = i+1; j<=n-1; j++)   // or j < n
                {
                        if(compare(a[pos], a[j]))
                        {       pos = j;                }
                }
                if(pos != i)
                {
                        int t = a[pos];  a[pos] = a[i];   a[i] = t;   //swap(&a[pos], &a[i]);
                }
        }
}
```

```
// Function to print an array
void printArray(int *arr, int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

//Client code
int main()
{       int a[ ] = {22,19,14,25,99,81};
        int n = sizeof(a)/sizeof(*a);
        printf("Before sorting\n");
        printArray(a, n);
        // Sorting in ascending order
        sort(a, n, ascending);
        printf("Ascending Order: ");
        printArray(a, n);
        // Sorting in descending order
        sort(a, n, descending);
        printf("Descending Order: ");
        printArray(a, n);
        return 0;
}
```

```
C:\Users\Dell>a
Before sorting
22 19 14 25 99 81
Ascending Order: 14 19 22 25 81 99
Descending Order: 99 81 25 22 19 14
```

Searching

In C, searching involves finding specific elements in an array based on a given condition. By using **callback functions, we can make the search process more flexible and reusable.** A **callback function allows us to define different search criteria dynamically without modifying the core search function.**

Coding Example_3: Implement the Binary Search on a sorted array of integers using a callback with the below constraints.

A. Search for a number if the number is even .

B. Search for a number if the number is less than 22.

```
#include<stdio.h>
int my_search(int *a,int low, int high,int key,int(*p)(int));
int is_even(int x);
int is_less_than_22(int x);
int main()
{       int a[] = {11,13,18,19,22,33,44,53,56,101};
        int n = sizeof(a)/sizeof(*a);
        printf("enter the element to be searched\n");
        int key;
        scanf("%d",&key);

        // perform search on a collection of elements and print only if it is even
        int pos = my_search(a,0,n-1,key,is_even);
        if(pos !=-1)
                printf("It is even and found at position: %d\n",pos);
        else
                printf("Not even. So not found\n");
        // perform search on a collection of elements and print only if it is less than 22
        pos = my_search(a,0,n-1,key,is_less_than_22);
        if(pos !=-1)
                printf("It is less than 22 and found at %d position\n",pos);
```

```
C:\Users\Dell>a
enter the element to be searched
45
Not even. So not found
Not less than 22. So not found

C:\Users\Dell>a
enter the element to be searched
44
It is even and found at position: 6
Not less than 22. So not found

C:\Users\Dell>a
enter the element to be searched
18
It is even and found at position: 2
It is less than 22 and found at 2 position
```

```
        else
                printf("Not less than 22. So not found\n");
        return 0;
}
int is_even(int x)
{       return x%2 == 0;}
int is_less_than_22(int x)
{       return x<22;}
int my_search(int a[],int low,int high,int key,int(*p)(int))
{       int pos = -1;    int mid;
        if (low>high)           return pos;
        else
        {       mid = (low+high)/2;
                if(a[mid]==key && p(key))  // very important
                {               pos = mid;      }
                else if(a[mid]>key)
                {               return my_search(a,low,mid-1,key,p);        }
                else
                {               return my_search(a,mid+1,high,key,p);       }
        }
        return pos;
}
```

A few problems for you to solve.

1. A teacher wants to sort a list of students based on their marks. The sorting should be flexible, allowing sorting in **ascending** (lowest to highest marks) or **descending** (highest to lowest marks) order **based on user preference.** Your task is to implement a **sorting function using a callback** to dynamically decide the sorting order.

2. Write a generic function that can compute the **maximum value, minimum value and the sum of all numbers** using **a callback function**. Test this generic function by passing an array of integers.

3. Given an array of integers, filter out only **even** or **odd** numbers dynamically based on a callback function. Define a function **filterArray()** that applies a **callback function** to filter numbers.

4. Find a number in an array based on different conditions:
   - **Find the first number greater than a given value.**
   - **Find the first number that is even.**
   - **Find the first number that is a multiple of 3.**

   Implement a generic function **findNumber()** that uses a **callback** to check conditions.

5. Implement a program that applies a transformation function to each element in an array. Transformations include:
   - **Squaring each number**
   - **Doubling each number**
   - **Taking the absolute value**

Happy Coding using Callback for Counting, Sorting and Searching!!